

Otomachi Una C++ 模板库

XCPC 考场代码手册

Junlin Ye

2026-09-29

按保留清单收录 40 份模板。编号与清单一致；正文给出调用方式、边界和代码。

考场版取舍：保留原有算法复杂度，缩短实现与重复部分。NTT 使用缓存根表的普通蝶形，MTT 使用三模 NTT 和 CRT；SA 使用计数排序倍增，LCP 复用四毛子 RMQ。可撤销并查集自带回滚记录，不再依赖单独的变量回滚模板。

共享代码：09 的固定模数类与阶乘表只抄一次。52 分为通用 NTT、998244353 卷积、公共 FPS 三段。使用 55 时，把 09 的 MOD 改成目标模数，保留 52 的通用 NTT 和公共 FPS，用 55 替换中间的卷积函数。每次只保留一个多项式卷积函数。

多项式范围：卷积、逆、 $\ln/\exp$ 与递推第 k 项保留快速算法；BM 和普通插值沿用考场版的二次算法。次数与模数的要求见对应节。

使用方式：GNU C++17/20。先抄 01 的公共底稿，再按对应节说明抄依赖和模板。行号不属于代码；排版换行不代表源码换行。各模板独立使用，同名模板不要一起粘贴。

目录

1	基础与数值类型	3
1.1	basic/src.cpp	3
1.2	basic/ST table (linear).cpp	5
1.3	basic/DSU.cpp	6
1.4	basic/BigInt.cpp	7
1.5	basic/Coordinate Compression.cpp	10
1.6	basic/ModInt.cpp	11
1.7	basic/ModInt-nonstaticMod.cpp	13
1.8	basic/DSU (with undo).cpp	15
2	数据结构	17
2.1	ds/zkw.cpp	17
2.2	ds/Dynamic Segment Tree.cpp	18
2.3	ds/lichao.cpp	20
2.4	ds/pq2stack.cpp	21
2.5	ds/xihe_tree.cpp	23
3	图论	26
3.1	graph/flow.cpp	26
3.2	graph/mcmf.cpp	28
3.3	graph/2-sat.cpp	31
3.4	graph/block-cut-tree.cpp	32
3.5	graph/blossom.cpp	34
3.6	graph/bipartite-matching.cpp	35
3.7	graph/directed-mst.cpp	37
3.8	graph/chordal.cpp	39
4	树上算法	42
4.1	trees/cartesian.cpp	42
5	数论	43
5.1	number theory/basic.cpp	43
5.2	number theory/pollard-rho.cpp	44
5.3	number theory/floor sum.cpp	47
5.4	number theory/du jiao sieve.cpp	47
6	多项式	50
6.1	polynomial/poly-fast.cpp	50
6.2	polynomial/MTT.cpp	53
6.3	polynomial/FWT.cpp	55
7	字符串	58
7.1	string/sa.cpp	58

7.2	string/sam.cpp	60
7.3	string/ac.cpp	62
7.4	string/gsam.cpp	63
7.5	string/kmp.cpp	65
7.6	string/hash.cpp	66
7.7	string/pam.cpp	68
7.8	string/manacher.cpp	69
7.9	string/runs.cpp	70
7.10	string/lyndon.cpp	72
8	计算几何	73
8.1	geometry/geometry-main.cpp	73

1 基础与数值类型

1.1 basic/src.cpp

01 公共底稿

什么时候用

提供类型别名、调试宏和其他模板依赖的辅助函数。自带标准头文件和 main，可直接作为提交程序的底稿。

调用顺序与接口

- 直接复制本文件，在 main 前粘贴算法，并在其中填写输入输出与求解逻辑。
- debug(...) / debugl(tag,...) 接受 1 至 10 个参数；编译时加 -DDEBUG 才输出，关闭时参数不求值。
- chkmin/chkmax 更新最值并返回是否修改。first_bit/last_bit 返回最低/最高置位的位置，参数必须非零。
- inf<int>、inf<ll> 已特化；其他类型默认值为零，不能当无穷大。

最小示例（放入 main）：

```
1 int a=7; chkmin(a,3);
2 assert(a==3 && first_bit(12)==2 && last_bit(12)==3);
3 debug(a);
4 cerr << "diagnostic\n";
```

使用边界

定义 LOCAL 时从 Otomachi_Una.in 读入并写入 Otomachi_Una.out，打开失败返回 1；否则使用标准输入输出。DEBUG 独立控制调试输出。未定义时通过宏屏蔽裸写的 cerr，不要写 std::cerr。向量输入前必须先分配非空长度；向量输出带方括号并截断长向量，正式答案应自己循环输出。

完整代码

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 using ll=long long;
4 using ull=unsigned long long;
5 // head file
6 #ifndef DEBUG
7     #define cerr for(;;false;) cerr
8 #endif
9 template<typename T>
10 inline constexpr T inf=T{};
11 template<>
12 inline constexpr int inf<int> = 1010000000;
13 template<>
14 inline constexpr long long inf<long long> = 2020000000000000000LL;
15 template<typename T,size_t S>
16 istream& operator >>(istream &o,array<T,S> &I){
17     assert(S>0);
18     for(int i=0;i<S;i++) o>>I[i];
19     return o;
20 }
21 template<typename T,size_t S>
22 ostream& operator <<(ostream &o,const array<T,S> &I){
23     o<<"[";
24     for(int i=0;i<S;i++){
25         if(i==50){o<<"...(total "<<S<<" elements)";break;}
26         o<<I[i];
```

```

27     if(i+1<I.size()) o<<" ";
28 }
29 o<<" ";
30 return o;
31 }
32 template<typename T>
33 istream& operator >>(istream &o,vector<T> &I){
34     assert(!I.empty());
35     for(int i=0;i<I.size();i++) o>>I[i];
36     return o;
37 }
38 template<typename T>
39 ostream& operator <<(ostream &o,const vector<T> &I){
40     o<<"[";
41     for(int i=0;i<I.size();i++){
42         if(i==50){o<<"...(total "<<I.size()<<" elements)";break;}
43         o<<I[i];
44         if(i+1<I.size()) o<<" ";
45     }
46     o<<"]";
47     return o;
48 }
49
50 #define DEBUG_1(x) #x"<<(x)
51 #define DEBUG_2(x,y) DEBUG_1(x)<<" , "<<DEBUG_1(y)
52 #define DEBUG_3(x,y,z) DEBUG_2(x,y)<<" , "<<DEBUG_1(z)
53 #define DEBUG_4(x,y,z,w) DEBUG_3(x,y,z)<<" , "<<DEBUG_1(w)
54 #define DEBUG_5(x,y,z,w,a) DEBUG_4(x,y,z,w)<<" , "<<DEBUG_1(a)
55 #define DEBUG_6(x,y,z,w,a,b) DEBUG_5(x,y,z,w,a)<<" , "<<DEBUG_1(b)
56 #define DEBUG_7(x,y,z,w,a,b,c) DEBUG_6(x,y,z,w,a,b)<<" , "<<DEBUG_1(c)
57 #define DEBUG_8(x,y,z,w,a,b,c,d) DEBUG_7(x,y,z,w,a,b,c)<<" , "<<DEBUG_1(d)
58 #define DEBUG_9(x,y,z,w,a,b,c,d,e) DEBUG_8(x,y,z,w,a,b,c,d)<<" , "<<DEBUG_1(e)
59 #define DEBUG_10(x,y,z,w,a,b,c,d,e,f) DEBUG_9(x,y,z,w,a,b,c,d,e)<<" , "<<DEBUG_1(f)
60 #define GET_MACRO(_1,_2,_3,_4,_5,_6,_7,_8,_9,_10,NAME,...) NAME
61 #ifdef DEBUG
62 #define debug(...) do { \
63     cerr<<"(line "<< __LINE__ <<"): "; \
64     cerr<<GET_MACRO(__VA_ARGS__,DEBUG_10,DEBUG_9,DEBUG_8, \
65         DEBUG_7,DEBUG_6,DEBUG_5,DEBUG_4, \
66         DEBUG_3,DEBUG_2,DEBUG_1)(__VA_ARGS__); \
67     cerr<<endl; \
68 } while(0)
69 #define debugl(s,...) do { \
70     cerr<<"(line "<< __LINE__ <<"): <<s<<" "; \
71     cerr<<GET_MACRO(__VA_ARGS__,DEBUG_10,DEBUG_9,DEBUG_8, \
72         DEBUG_7,DEBUG_6,DEBUG_5,DEBUG_4, \
73         DEBUG_3,DEBUG_2,DEBUG_1)(__VA_ARGS__); \
74     cerr<<endl; \
75 } while(0)
76 #else
77 #define debug(...) ((void)0)
78 #define debugl(...) ((void)0)
79 #endif
80 inline int last_bit(ull x){assert(x);return 63-__builtin_clzll(x);}
81 inline int first_bit(ull x){assert(x);return __builtin_ctzll(x);}

```

```

82 inline int popc(ull x){return __builtin_popcountll(x);}
83 template<typename A,typename B> inline bool chkmax(A &x,const B &y){if(x<y){
84     x=y;return true;}return false;}
85 template<typename A,typename B> inline bool chkmin(A &x,const B &y){if(x>y){
86     x=y;return true;}return false;}
87 int main(){
88     #ifdef LOCAL
89         if(!freopen("Otomachi_Una.in","r",stdin)) return 1;
90         if(!freopen("Otomachi_Una.out","w",stdout)) return 1;
91     #endif
92     ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
93     return 0;
94 }

```

1.2 basic/ST table (linear).cpp

03 四毛子 RMQ：线性预处理与空间

什么时候用

静态数组最值。采用块内单调栈位掩码、块间 ST 表；在机器字位运算为 $O(1)$ 的模型下，预处理及空间 $O(n)$ ，查询 $O(1)$ 。

调用顺序与接口

`Linear_RMQ<int> t; t.build(a)` 接受从 0 开始的 vector，内部保存副本。`ask(l,r)` 查询闭区间最小值，`pos(l,r)` 返回最小值下标；相等取最左。`Linear_RMQ<int,true>` 改为最大值，其他接口相同。

最小示例（放入 main）：

```

1 vector<int> a{4,1,1,3};
2 Linear_RMQ<int> mn; mn.build(a);
3 assert(mn.ask(0,3)==1 && mn.pos(0,3)==1);
4 Linear_RMQ<int,true> mx; mx.build(a);
5 assert(mx.ask(1,3)==3);

```

使用边界

块长 $B=\max(1,\text{floor}(\log_2 n))$ ，不是固定常数；块数 $m=\text{ceil}(n/B)$ ，块间表 $O(m \log m)=O(n)$ 。每个元素存一份块前缀的单调栈掩码，一次清除低位和 `ctz` 即可查询块内最左最值。 $n<\text{INT_MAX}$ ，块长最多 30，使用 unsigned 位运算。可 build 空数组，但不能查询；修改数组后须重新 build。`set()` 清空。不需要普通 ST 表或笛卡尔树作为依赖。

完整代码

```

1 template<typename T,bool MAX=false> struct Linear_RMQ{
2 private:
3     int n=0,B=1;
4     vector<T> a;
5     vector<unsigned> mask;
6     vector<vector<int>>> st;
7     bool better(int x,int y)const{if constexpr(MAX)return a[y]<a[x];else return
8         a[x]<a[y];}
9     int chk(int x,int y)const{return better(y,x)?y:x;}
10    int small(int l,int r)const{
11        unsigned s=mask[r]&(~0u<<(1%B));
12        return r/B*B+__builtin_ctz(s);
13    }
14 public:

```

```

15 void set(){n=0;B=1;a.clear();mask.clear();st.clear();}
16 void build(const vector<T> &v){
17     assert(v.size()<(size_t)INT_MAX);a=v;n=a.size();mask.resize(n);st.clear();
18     if(!n){B=1;return;}
19     // B grows with log(n); the block sparse table has only O(n) entries.
20     B=max(1,31-__builtin_clz((unsigned)n));
21     int m=(n-1)/B+1;st.resize(32-__builtin_clz((unsigned)m));st[0].resize(m);
22     unsigned s=0;
23     for(int i=0;i<n;i++){
24         if(i%B==0)s=0;
25         while(s){
26             int j=31-__builtin_clz(s);
27             if(!better(i,i/B*B+j))break;
28             s^=1u<<j;
29         }
30         mask[i]=s|1u<<(i%B);
31         st[0][i/B]=i/B*B+__builtin_ctz(s);
32     }
33     for(int k=1;k<(int)st.size();k++){
34         st[k].resize(m-(1<<k)+1);
35         for(int i=0;i<(int)st[k].size();i++)st[k][i]=chk(st[k-1][i],
36             st[k-1][i+(1<<(k-1))]);
37     }
38 }
39 int pos(int l,int r)const{
40     assert(0<=l&&l<=r&&r<n);
41     int x=l/B,y=r/B;if(x==y)return small(l,r);
42     int ans=small(l,(x+1)*B-1);
43     if(x+1<y){
44         int k=31-__builtin_clz((unsigned)(y-x-1));
45         ans=chk(ans,chk(st[k][x+1],st[k][y-(1<<k)]));
46     }
47     return chk(ans,small(y*B,r));
48 }
49 T ask(int l,int r)const{return a[pos(l,r)];}
50 };

```

1.3 basic/DSU.cpp

04 普通并查集

什么时候用

维护集合连通关系，路径压缩配合按大小合并。操作均摊近似 $O(1)$ ，空间 $O(n)$ 。

调用顺序与接口

- DSU d; d.setN(n) 创建 1..n 的独立集合。
- find(x) 返回代表元; same(x,y) 判断同集合。
- merge(x,y) 成功返回新代表元，已经连通返回 0。siz[find(x)] 为集合大小。

最小示例（放入 main）:

```

1 DSU d; d.setN(4);
2 d.merge(1,2); assert(d.same(1,2));
3 assert(d.siz[d.find(1)]==2 && !d.same(1,3));

```

使用边界

再次 setN 会清空关系。普通版不能回滚；需要撤销时选择 unionfind。

完整代码

```

1 struct DSU{
2   int n=0;
3   vector<int> fa,siz;
4   void setN(int _n){
5     assert(0<=_n&&_n<INT_MAX);n=_n;
6     fa.resize(n+1);iota(fa.begin(),fa.end(),0);siz.assign(n+1,1);siz[0]=0;
7   }
8   inline int find(int x){
9     assert(1<=x&&x<=n);
10    while(x^fa[x])x=fa[x]=fa[fa[x]];return x;
11  }
12  inline int merge(int x,int y){
13    assert(1<=min(x,y)&&max(x,y)<=n);
14    x=find(x),y=find(y);
15    if(x==y) return 0;
16    if(siz[x]<siz[y]) swap(x,y);
17    siz[x]+=siz[y],fa[y]=x;return x;
18  }
19  inline bool same(int x,int y){
20    assert(1<=min(x,y)&&max(x,y)<=n);
21    return find(x)==find(y);
22  }
23 };

```

1.4 basic/BigInt.cpp

06 高精度有符号整数

什么时候用

十进制大整数，vector 按 10^9 分块。加减 $O(n)$ ，普通乘法及长除法最坏 $O(nm)$ ， n 、 m 是分块数。

调用顺序与接口

- BigInt x=123 从 long long 构造；更长整数用流读入。
- 支持 + - * / %、复合赋值和比较。
- a.divmod(b) 同时返回商和余数，避免分别执行除法与取模。

最小示例（放入 main）：

```

1 BigInt a,b;
2 istream in("-12345678901234567890 100");
3 in >> a >> b;
4 auto [q,r]=a.divmod(b);
5 assert(q*b+r==a);
6 ostream out; out << q << ' ' << r;
7 assert(out.str()=="-123456789012345678 -90");

```

使用边界

除法向零取整，余数与被除数同号；除数不能为零。这不是 FFT 高精度乘法，也不支持小数。

完整代码

```

1 struct BigInt{
2     static constexpr int base=1000000000;
3     vector<int> a;
4     int sign=1;
5     BigInt(long long x=0){
6         unsigned long long y=x;
7         if(x<0)sign=-1,y=0-y;
8         for(;y/=base)a.push_back(y%base);
9     }
10    void trim(){while(!a.empty()&&!a.back())a.pop_back();if(a.empty())sign=1;}
11    explicit operator bool()const{return !a.empty();}
12    int cmp_abs(const BigInt &b)const{
13        if(a.size()!=b.a.size())return a.size()<b.a.size()?-1:1;
14        for(int i=(int)a.size()-1;i>=0;i--)if(a[i]!=b.a[i])return
15            a[i]<b.a[i]?-1:1;
16        return 0;
17    }
18    int cmp(const BigInt &b)const{return
19        sign!=b.sign?(sign<b.sign?-1:1):sign*cmp_abs(b);}
20    bool operator <(const BigInt &b)const{return cmp(b)<0;}
21    bool operator >(const BigInt &b)const{return cmp(b)>0;}
22    bool operator <=(const BigInt &b)const{return cmp(b)<=0;}
23    bool operator >=(const BigInt &b)const{return cmp(b)>=0;}
24    bool operator==(const BigInt &b)const{return sign==b.sign&& a==b.a;}
25    bool operator!=(const BigInt &b)const{return !(*this==b);}
26    BigInt operator-()const{BigInt b=*this;if(b)b.sign=-b.sign;return b;}
27    BigInt operator+()const{return *this;}
28    BigInt operator+(const BigInt &b)const{
29        if(!b)return *this;
30        if(!*this)return b;
31        if(sign!=b.sign)return *this-(-b);
32        BigInt c;c.sign=sign;c.a.resize(max(a.size(),b.a.size()));
33        int carry=0;
34        for(int i=0;i<(int)c.a.size();i++){
35            long long x=(long
36                long)carry+(i<(int)a.size()?a[i]:0)+(i<(int)b.a.size()?b.a[i]:0);
37            c.a[i]=x%base;carry=x/base;
38        }
39        if(carry)c.a.push_back(carry);
40        return c;
41    }
42    BigInt operator-(const BigInt &b)const{
43        if(!b)return *this;
44        if(!*this)return -b;
45        if(sign!=b.sign)return *this+(-b);
46        if(cmp_abs(b)<0)return -(b-*this);
47        BigInt c=*this;int carry=0;
48        for(int i=0;i<(int)a.size();i++){
49            int x=a[i]-carry-(i<(int)b.a.size()?b.a[i]:0);
50            carry=x<0;c.a[i]=x+(carry?base:0);
51        }
52        c.trim();return c;
53    }
54    BigInt mul(int x)const{

```

```

55     assert(0<=x&&x<base);
56     BigInt c;c.sign=sign;c.a.resize(a.size());long long carry=0;
57     for(int i=0;i<(int)a.size();i++){
58         long long y=1ll*a[i]*x+carry;c.a[i]=y%base;carry=y/base;
59     }
60     if(carry)c.a.push_back(carry);
61     c.trim();return c;
62 }
63 BigInt operator*(const BigInt &b)const{
64     BigInt c;if(!*this||!b)return c;
65     c.sign=sign*b.sign;c.a.resize(a.size()+b.a.size());
66     for(int i=0;i<(int)a.size();i++){
67         long long carry=0;
68         for(int j=0;j<(int)b.a.size();j++){
69             long long x=c.a[i+j]+1ll*a[i]*b.a[j]+carry;
70             c.a[i+j]=x%base;carry=x/base;
71         }
72         c.a[i+b.a.size()]=carry;
73     }
74     c.trim();return c;
75 }
76 // Division truncates towards zero; remainder has the dividend's sign.
77 pair<BigInt,BigInt> divmod(const BigInt &b)const{
78     assert(b);
79     if(cmp_abs(b)<0)return {0,*this};
80     if(b.a.size()==1){
81         BigInt q=*this;long long rem=0;
82         for(int i=(int)a.size()-1;i>=0;i--){
83             long long x=rem*base+a[i];q.a[i]=x/b.a[0];rem=x%b.a[0];
84         }
85         q.sign=sign*b.sign;q.trim();return {q,BigInt(rem*sign)};
86     }
87     int norm=base/(b.a.back()+1);
88     BigInt x=mul(norm),y=b.mul(norm),q,r;x.sign=y.sign=1;
89     q.a.resize(x.a.size());
90     for(int i=(int)x.a.size()-1;i>=0;i--){
91         r.a.insert(r.a.begin(),x.a[i]);r.trim();
92         int n=y.a.size();
93         long long hi=r.a.size()>(size_t)n?r.a[n]:0;
94         long long lo=r.a.size()>=(size_t)n?r.a[n-1]:0;
95         int d=min((hi*base+lo)/y.a.back(),(long long)base-1);
96         r=r-y.mul(d);
97         while(r.sign<0)r=r+y,--d;
98         q.a[i]=d;
99     }
100     long long carry=0;
101     for(int i=(int)r.a.size()-1;i>=0;i--){
102         long long x=r.a[i]+carry*base;r.a[i]=x/norm;carry=x%norm;
103     }
104     q.sign=sign*b.sign;r.sign=sign;q.trim();r.trim();return {q,r};
105 }
106 BigInt operator/(const BigInt &b)const{return divmod(b).first;}
107 BigInt operator%(const BigInt &b)const{return divmod(b).second;}
108 BigInt& operator+=(const BigInt &b){return *this=*this+b;}
109 BigInt& operator-=(const BigInt &b){return *this=*this-b;}

```

```

110 BigInt& operator*=(const BigInt &b){return *this=*this*b;}
111 BigInt& operator/=(const BigInt &b){return *this=*this/b;}
112 BigInt& operator %=(const BigInt &b){return *this=*this%b;}
113 friend istream& operator >>(istream &in,BigInt &x){
114     string s;if(!(in>>s))return in;
115     int l=(s[0]=='+'||s[0]=='-');
116     if(l==(int)s.size()){in.setstate(ios::failbit);return in;}
117     for(int i=l;i<(int)s.size();i++)if(s[i]<'0'||s[i]>'9'){
118         in.setstate(ios::failbit);return in;
119     }
120     BigInt y;y.sign=s[0]=='-'?-1:1;y.a.reserve((s.size()-l+8)/9);
121     for(int r=s.size();r>l;r-=9){
122         int v=0;for(int i=max(l,r-9);i<r;i++)v=v*10+s[i]-'0';
123         y.a.push_back(v);
124     }
125     y.trim();x=move(y);return in;
126 }
127 friend ostream& operator <<(ostream &out,const BigInt &x){
128     if(!x)return out<<'0';
129     string s=x.sign<0?"-":"";s+=to_string(x.a.back());
130     for(int i=(int)x.a.size()-2;i>=0;i--){
131         string t=to_string(x.a[i]);s.append(9-t.size(),'0');s+=t;
132     }
133     return out<<s;
134 }
135 };

```

1.5 basic/Coordinate Compression.cpp

08 离散化并原地回写

什么时候用

先收集所有会用到的值，再排序去重，把已登记变量改写成 1..k 的排名。时间 $O(n \log n)$ 。

调用顺序与接口

- `compress<T> c; c.set()` 清空。`c.pb(x)` 要求可修改左值，会保存 `x` 的地址。
- `c.build()` 原地改写 `x`；`c.id(original)` 找已登记原值的排名。
- `c[i]` 由排名恢复原值；`range(l,r)` 返回落在原值闭区间中的排名范围，左端大于右端表示空。

最小示例（放入 main）：

```

1 compress<int> c; c.set();
2 int a=30,b=10,d=30;
3 c.pb(a); c.pb(b); c.pb(d); c.build();
4 assert(a==2 && b==1 && d==2 && c[2]==30);
5 assert(c.id(30)==2);

```

使用边界

`pb` 后、`build` 前变量必须存活且地址稳定：不要登记即将离开作用域的局部变量，也不要让所属 `vector` 扩容。`id` 参数是原值，不是已改写后的排名。`range` 当前端点参数为 `int`。

完整代码

```

1  template<typename T>
2  struct compress{
3  vector<T> I;
4  vector<T*> II;
5  // make compress values to 1 ~ size
6  void set(){I.clear();II.clear();}
7  void pb(T &x){I.push_back(x);II.push_back(&x);}
8  void pb(const T &x){I.push_back(x);}
9  int id(T x)const{
10   auto it=lower_bound(I.begin(),I.end(),x);
11   assert(it!=I.end()&&*it==x);
12   return it-I.begin()+1;
13 }
14 void build(){
15   sort(I.begin(),I.end());
16   I.erase(unique(I.begin(),I.end()),I.end());
17   vector<int> a; a.reserve(II.size());
18   for(auto p:II)a.push_back(id(*p));
19   for(int i=0;i<(int)II.size();i++)*II[i]=a[i];
20   II.clear();
21 }
22 int size()const{return I.size();}
23 array<int,2> range(const T &l,const T &r)const{
24   int a=lower_bound(I.begin(),I.end(),l)-I.begin();
25   int b=upper_bound(I.begin(),I.end(),r)-I.begin();
26   return {a+1,b};
27 }
28 T operator [](int x)const{
29   assert(1<=x&&x<=I.size());
30   return I[x-1];
31 }
32 };
33 // !!!!!!!!!!! before each use, use cc.set() to clear !!!!!!!!!!!
34 // !!!!!!!!!!! 如果元素被插入, cc.build() 会自动重编号 !!!!!!!!!!!

```

1.6 basic/ModInt.cpp

09 固定模数及组合数

什么时候用

默认模数 998244353。模数写在源码中的 MOD；阶乘表按需扩容。

调用顺序与接口

- mint 支持四则、pow(k)、inv()；除数必须可逆。
- init(n) 保证 fac、ifac、inv 覆盖到 n，组合数调用会自动预处理。
- C(k,n) 和 binom(n,k) 都表示从 n 个中选 k 个；这里 C 的参数顺序与常见记法相反。

最小示例（放入 main）：

```

1  init(10);
2  assert(C(2,5).x==10 && binom(5,2).x==10);
3  mint a=3; assert((a*a.inv()).x==1);

```

```
4 assert(a.pow(4).x==81);
```

使用边界

模数须在 $2..INT_MAX$ 。阶乘表要求质数模数且 $n < MOD$ ；逆元要求与模数互质。支持 $==/!=$ 、负数输入、ull 构造与 $pow(LLONG_MIN)$ 。多项式共用本节代码，只抄一次。

完整代码

```
1 template<unsigned M>
2 struct ModInt{
3     static constexpr unsigned MOD=M;
4     static_assert(1<M&&M<=INT_MAX);
5     unsigned x=0;
6     ModInt()=default;
7     template<typename T> ModInt(T v){
8         if constexpr(is_signed_v<T>){ll y=v%(ll)M;x=y<0?y+M:y;}
9         else x=v%M;
10    }
11    ModInt &operator+=(ModInt a){if((x+=a.x)>=M)x-=M;return *this;}
12    ModInt &operator-=(ModInt a){if((x-=a.x)>=M)x+=M;return *this;}
13    ModInt &operator*=(ModInt a){x=(ull)x*a.x%M;return *this;}
14    ModInt &operator/=(ModInt a){return *this**a.inv();}
15    friend ModInt operator+(ModInt a,ModInt b){return a+=b;}
16    friend ModInt operator-(ModInt a,ModInt b){return a-=b;}
17    friend ModInt operator*(ModInt a,ModInt b){return a*=b;}
18    friend ModInt operator/(ModInt a,ModInt b){return a/=b;}
19    bool operator==(ModInt a)const{return x==a.x;}
20    bool operator!=(ModInt a)const{return x!=a.x;}
21    explicit operator bool()const{return x!=0;}
22    bool operator!()const{return !x;}
23    ModInt operator+()const{return *this;}
24    ModInt operator-()const{ModInt a;a.x=x?M-x:0;return a;}
25    ModInt &operator++(){return *this+=1;}
26    ModInt &operator--(){return *this-=1;}
27    ModInt operator++(int){ModInt a=*this;++*this;return a;}
28    ModInt operator--(int){ModInt a=*this;--*this;return a;}
29    ModInt inv()const{
30        ll a=M,b=x,y=0,z=1;
31        while(b){ll q=a/b;a-=q*b;swap(a,b);y-=q*z;swap(y,z);}
32        assert(a==1);return y;
33    }
34    ModInt pow(ll k)const{
35        ull e=k;ModInt a=*this,b=1;
36        if(k<0){a=a.inv();e=0-e;}
37        for(;e>=1,a*=a)if(e&1)b*=a;
38        return b;
39    }
40    friend istream &operator>>(istream &o,ModInt &a){
41        ll x;if(o>>x)a=x;return o;
42    }
43    friend ostream &operator<<(ostream &o,ModInt a){return o<<a.x;}
44 };
45
46 const int MOD=998244353;
47 using mint=ModInt<MOD>;
48 vector<mint> fac{1},ifac{1},inv{0,1};
```

```

49 void init(int n=0){
50     assert(0<=n&&n<MOD);
51     int m=fac.size();if(n<m)return;
52     n=max(n,(int)min(211*m,(11)MOD-1));
53     fac.resize(n+1);ifac.resize(n+1);inv.resize(max(n+1,2));
54     for(int i=m;i<=n;i++){
55         fac[i]=fac[i-1]*i;
56         if(i>1)inv[i]=- (MOD/i)*inv[MOD%i];
57         ifac[i]=ifac[i-1]*inv[i];
58     }
59 }
60 inline mint C(int x,int y){
61     // choose x from y
62     if(x<0||y<x)return 0;
63     init(y);return fac[y]*ifac[x]*ifac[y-x];
64 }
65 inline mint binom(int y,int x){return C(x,y);}
66
67 inline mint Apple_in_Box(int n,int m){
68     // n apples into m boxes, each box must be non-empty
69     return C(m-1,n-1);
70 }
71 inline mint Apple_in_Box(int n,int m,int k){
72     // n apples into m boxes,
73     // the first k boxes must be non-empty (others can be empty)
74     assert(0<=k&&k<=m);
75     return Apple_in_Box(n+(m-k),m);
76 }

```

1.7 basic/ModInt-nonstaticMod.cpp

10 运行时模数及组合数

什么时候用

同一份程序需要选定不同模数时使用。设置模数后再创建该轮使用的值。

调用顺序与接口

- `mint::setM(p)` 修改模数。
- `init_fact(n)` 重建 $0..n$ 阶乘、逆阶乘，要求 $n!$ 可逆。
- `C(k,n)` / `binom(n,k)` 查询前必须已预处理到 n 。

最小示例（放入 main）：

```

1 mint::setM(101); init_fact(10);
2 assert(C(2,5).x==10);
3 mint a=100; assert((a+2).x==1);

```

使用边界

修改模数不会转换已有 `mint` 变量；丢弃旧值并重建阶乘。与固定模数版、各多项式版本的 `mint` / `fac` 等全局名字冲突。

完整代码

```

1 struct mint{
2 static inline unsigned M=998244353;
3 static inline ull R=-1ull/M;
4 static void setM(unsigned m){assert(1<m&&m<=INT_MAX);M=m;R=-1ull/M;}
5 unsigned x=0;
6 mint()=default;
7 template<typename T> mint(T v){
8 if constexpr(is_signed_v<T>){ll y=v%(ll)M;x=y<0?y+M:y;}
9 else x=v%M;
10 }
11 mint &operator+=(mint a){if((x+=a.x)>=M)x-=M;return *this;}
12 mint &operator-=(mint a){if((x-=a.x)>=M)x+=M;return *this;}
13 mint &operator*=(mint a){
14 ull v=(ull)x*a.x,q=(__uint128_t)R*v>>64;
15 v-=q*M;x=v>=M?v-M:v;return *this;
16 }
17 mint &operator/=(mint a){return *this*=a.inv();}
18 friend mint operator+(mint a,mint b){return a+=b;}
19 friend mint operator-(mint a,mint b){return a-=b;}
20 friend mint operator*(mint a,mint b){return a*=b;}
21 friend mint operator/(mint a,mint b){return a/=b;}
22 bool operator==(mint a)const{return x==a.x;}
23 bool operator!=(mint a)const{return x!=a.x;}
24 explicit operator bool()const{return x!=0;}
25 bool operator!()const{return !x;}
26 mint operator+()const{return *this;}
27 mint operator-()const{mint a;a.x=x?M-x:0;return a;}
28 mint &operator++(){return *this+=1;}
29 mint &operator--(){return *this-=1;}
30 mint operator++(int){mint a=*this;++*this;return a;}
31 mint operator--(int){mint a=*this;--*this;return a;}
32 mint inv()const{
33 ll a=M,b=x,y=0,z=1;
34 while(b){ll q=a/b;a-=q*b;swap(a,b);y-=q*z;swap(y,z);}
35 assert(a==1);return y;
36 }
37 mint pow(ll k)const{
38 ull e=k;mint a=*this,b=1;
39 if(k<0){a=a.inv();e=0-e;}
40 for(;e>=1;a*=a)if(e&1)b*=a;
41 return b;
42 }
43 friend istream &operator>>(istream &o,mint &a){
44 ll x;if(o>>x)a=x;return o;
45 }
46 friend ostream &operator<<(ostream &o,mint a){return o<<a.x;}
47 };
48
49 vector<mint> fac,ifac,inv;
50 unsigned fact_mod=0;
51 void init_fact(int n=0){
52 assert(0<=n&&(unsigned)n<mint::M);
53 fac.assign(n+1,1);ifac.resize(n+1);inv.assign(n+1,0);
54 for(int i=1;i<=n;i++)fac[i]=fac[i-1]*i;

```

```

55  ifac[n]=fac[n].inv();
56  for(int i=n;i>=1;i--)ifac[i-1]=ifac[i]*i;
57  for(int i=1;i<=n;i++)inv[i]=ifac[i]*fac[i-1];
58  fact_mod=mint::M;
59  }
60  inline mint C(int x,int y){
61  if(x<0||y<x)return 0;
62  assert(fact_mod==mint::M&&y<(int)fac.size());
63  return fac[y]*ifac[x]*ifac[y-x];
64  }
65  inline mint binom(int y,int x){return C(x,y);}

```

1.8 basic/DSU (with undo).cpp

13 可撤销并查集

什么时候用

适合线段树分治或搜索中加入连边后回退。按大小合并、不做路径压缩，单次查找 $O(\log n)$ 。

调用顺序与接口

`unionfind d`; `d.setN(n)` 初始化; `find/merge` 同普通版。`int t=d.version()` 保存快照, `d.roll_back(t)` 撤销到该快照。记录直接保存在并查集内, 无其他依赖。

最小示例 (放入 main):

```

1 unionfind d; d.setN(3);
2 auto t=d.version(); d.merge(1,2);
3 assert(d.find(1)==d.find(2));
4 d.roll_back(t); assert(d.find(1)!=d.find(2));

```

使用边界

`setN` 会清空历史, 旧快照随之失效。不要加入路径压缩; 单次 `find/merge` 为 $O(\log n)$, 每撤销一次实际合并为 $O(1)$ 。

完整代码

```

1 struct unionfind{
2  int n=0;
3  vector<int> fa,siz,h;
4  void setN(int m){
5      assert(0<=m&&m<INT_MAX);n=m;h.clear();
6      fa.resize(n+1);iota(fa.begin(),fa.end(),0);
7      siz.assign(n+1,1);siz[0]=0;
8  }
9  int find(int x)const{
10     assert(1<=x&&x<=n);
11     while(x!=fa[x])x=fa[x];
12     return x;
13  }
14  int merge(int x,int y){
15     x=find(x);y=find(y);if(x==y)return 0;
16     if(siz[x]<siz[y])swap(x,y);
17     h.push_back(y);fa[y]=x;siz[x]+=siz[y];return x;
18  }
19  int version()const{return h.size();}
20  void roll_back(int t){
21     assert(0<=t&&t<=(int)h.size());

```

```
22  while((int)h.size()>t){  
23      int y=h.back();h.pop_back();siz[fa[y]]-=siz[y];fa[y]=y;  
24  }  
25  }  
26  };
```

2 数据结构

2.1 ds/zkw.cpp

23 非递归线段树框架

什么时候用

单点赋值和闭区间聚合，当前 info 维护区间和与最大前缀和（允许空前缀）。单次 $O(\log n)$ 。

调用顺序与接口

- `zkw_segt t; t.setN(n)` 下标为 $0..n-1$ 。
- `upd(i,a,b)` 设置该叶 info 的 $s=a$ 、 $mx=b$ 。
- 合并两段时 $s=L.s+R.s$ ， $mx=\max(L.mx, L.s+R.mx)$ ；ask 返回最终 mx 。

最小示例（放入 main）：

```
1 zkw_segt t; t.setN(3);
2 t.upd(0,-4,0); t.upd(1,9,9); t.upd(2,-2,0);
3 assert(t.ask(0,2)==5);
```

使用边界

普通数组的叶子设为 $s=a[i]$ 、 $mx=\max(0,a[i])$ 。若 s 全为零且 mx 非负，也可用于区间最大值。其他统计应重写 info 的合并和单位元；合并必须满足结合律。下标从 0 开始。

完整代码

```
1 struct zkw_segt{
2 // information
3 struct info{
4 // sum and maximum prefix sum, including the empty prefix
5 ll s,mx;
6 info():s(0),mx(0){}
7 info(ll _s,ll _mx):s(_s),mx(_mx){}
8 inline info& operator+=(const info &y){
9 ll os=s;
10 s+=y.s;
11 mx=max(mx,os+y.mx);
12 return *this;
13 }
14 inline info operator+(const info &y)const{
15 return info(*this)+=y;
16 }
17 };
18 // basic segment tree template, private operations
19 private:
20 int n=0,B=0;vector<info> a;
21 void modify(int x,info v){
22 x+=B+1;
23 a[x]=v;
24 while(x>1) x>>=1,a[x]=a[x<<1]+a[x<<1|1];
25 }
26 info query(int l,int r){
27 l+=B,r+=B+2;
28 info ql,qr;
29 while(l^r^1){
30 if(~l&1) ql=ql+a[l^1];
```

```

31     if(r&1) qr=a[r^1]+qr;
32     l>>=1,r>>=1;
33 }
34 return ql+qr;
35 }
36 // basic segment tree template, public operations
37 // please transfer (int,int) to info here
38 public:
39 void setN(int _n){
40     assert(1<=_n&&_n<(1<<29));n=_n;B=1;
41     while(B<n+2)B*=2;
42     a.assign(2*B,info());
43 }
44 void upd(int x,ll a,ll b){
45     assert(0<=x&&x<n);
46     modify(x,info(a,b));
47 }
48 ll ask(int l,int r){
49     assert(0<=l&&l<=r&&r<n);
50     return query(l,r).mx;
51 }
52 };

```

2.2 ds/Dynamic Segment Tree.cpp

24 大值域稀疏单点统计

什么时候用

在整数键 $-10^9..10^9$ 上做单点加、区间和；节点带键交换以压缩空间。单次 $O(\log V)$ 。

调用顺序与接口

- `dynamic_segt t; t.set()` 清空；`set(m)` 可预留节点。
- `add(x,v)` 给键 `x` 的权值加 `v`；`ask(l,r)` 求键区间权值和。

最小示例（放入 `main`）：

```

1 dynamic_segt t; t.set();
2 t.add(-100,3); t.add(200,4);
3 assert(t.ask(-100,0)==3);

```

使用边界

空键权值为零。与主席树不同：不保留历史版本，也不支持复用旧根。文件有全局对象 `T`。

完整代码

```

1 struct dynamic_segt{
2 private:
3     static const int L=-1e9,R=1e9;
4     int tot=0;
5     struct info{
6         ll s;
7         info():s(0){}
8         info(ll _s):s(_s){}
9         inline info& operator+=(const info &x){s+=x.s;return *this;}
10        inline info operator+(const info &x)const{return info(*this)+=x;}

```

```

11 };
12 struct node{
13     int p=0,ls=0,rs=0;
14     info self,val;
15 };
16 vector<node> tr=vector<node>(1);
17 int rt=0;
18 int new_node(int p,info x){
19     assert(tr.size()<INT_MAX);tr.push_back({p,0,0,x,x});
20     return ++tot;
21 }
22 inline void pushup(int id){
23     tr[id].val=tr[tr[id].ls].val+tr[id].self+tr[tr[id].rs].val;
24 }
25 int add(int p,info v,int id,int l,int r){
26     if(!id)return new_node(p,v);
27     if(tr[id].p==p){tr[id].self+=v;pushup(id);return id;}
28     int mid=l+((ll)r-l)/2;
29     if(p<=mid){
30         if(p>tr[id].p)swap(tr[id].p,p),swap(tr[id].self,v);
31         int x=add(p,v,tr[id].ls,l,mid);tr[id].ls=x;
32     }else{
33         if(p<tr[id].p)swap(tr[id].p,p),swap(tr[id].self,v);
34         int x=add(p,v,tr[id].rs,mid+1,r);tr[id].rs=x;
35     }
36     pushup(id);return id;
37 }
38 void query(int ql,int qr,int id,int l,int r,info &X){
39     if(max(ql,l)>min(r,qr)||!id) return;
40     if(ql<=l&&r<=qr){X+=tr[id].val;return;}
41     int mid=l+((ll)r-l)/2;
42     query(ql,qr,tr[id].ls,l,mid,X);
43     if(ql<=tr[id].p&&tr[id].p<=qr) X+=tr[id].self;
44     query(ql,qr,tr[id].rs,mid+1,r,X);
45 }
46 public:
47 void set(int m=0){assert(m>=0);tot=rt=0;tr.assign(1,node());
48     tr.reserve((size_t)m+1);}
49 void add(int p,ll s){
50     assert(L<=p&&p<=R);
51     rt=add(p,info(s),rt,L,R);
52 }
53 ll ask(int l,int r){
54     assert(L<=l&&l<=r&&r<=R);
55     info X;
56     query(l,r,rt,L,R,X);
57     return X.s;
58 }
59 }T;

```

2.3 ds/lichao.cpp

26 线段李超树

什么时候用

插入只在一段整数横坐标上有效的直线，查询某点的最小值。

调用顺序与接口

- lichao t; t.set() 清空。
- add(line{k,b},l,r) 插入 $y=kx+b$; 省略 l,r 表示整个默认值域。
- ask(x) 返回最小纵坐标，没有覆盖返回 $2e18$ 。

最小示例（放入 main）：

```
1 lichao t; t.set();
2 t.add({2,3},0,10); t.add({-1,8},0,10);
3 assert(t.ask(2)==6);
```

使用边界

横坐标及斜率限制在 $[-10^9, 10^9]$ ，有效端点的纵坐标需在 $[-2e18, 2e18]$ 。整条线插入 $O(\log V)$ ，线段插入 $O(\log^2 V)$ ，查询 $O(\log V)$ 。

完整代码

```
1 struct line{
2     ll k,b;
3     ll f(ll x){return k*x+b;}
4     // please, check whether use __int128
5 };
6 struct lichao{
7     // you may add y=kb+x where x \in [l,r]
8     // you can query x, answer the **min** y (if no such, return inf = 2e18)
9     static const ll inf=2e18,V=1e9;
10    vector<int> ls{0},rs{0},p{0};int tot=0,cnt=0,rt=0;
11    vector<line> L{{0,inf}};
12    // if you don't use range add, this can up to n
13    inline ll gv(const int &id,const ll &v){return L[id].f(v);}
14    void set(){
15        ls.assign(1,0);rs.assign(1,0);p.assign(1,0);
16        L.assign(1,{0,inf});tot=cnt=rt=0;
17    }
18    int new_node(){
19        assert(tot<INT_MAX-1);ls.push_back(0);rs.push_back(0);p.push_back(0);
20        return ++tot;
21    }
22    int add(int x,int id,int l,int r){
23        if(!id)id=new_node();
24        if(l==r){
25            if(gv(x,l)<gv(p[id],l))p[id]=x;
26            return id;
27        }
28        int mid=l+((ll)r-l)/2;
29        if(gv(x,mid)<gv(p[id],mid))swap(x,p[id]);
30        if(gv(x,l)<gv(p[id],l)){int t=add(x,ls[id],l,mid);ls[id]=t;}
31        if(gv(x,r)<gv(p[id],r)){int t=add(x,rs[id],mid+1,r);rs[id]=t;}
32        return id;
```

```

33 }
34 int upd(int x,int ql,int qr,int id,int l,int r){
35     if(max(ql,l)>min(r,qr))return id;
36     if(!id)id=new_node();
37     if(ql<=l&&r<=qr)return add(x,id,l,r);
38     int mid=l+((ll)r-l)/2;
39     int a=upd(x,ql,qr,ls[id],l,mid),b=upd(x,ql,qr,rs[id],mid+1,r);
40     ls[id]=a;rs[id]=b;return id;
41 }
42 ll ask(int x,int id,int l,int r){
43     if(!id) return inf;
44     ll ans=gv(p[id],x);
45     if(l==r) return ans;
46     int mid=l+((ll)r-l)/2;
47     if(x<=mid) ans=min(ans,ask(x,ls[id],l,mid));
48     else ans=min(ans,ask(x,rs[id],mid+1,r));
49     return ans;
50 }
51
52 void add(line x,int l=-V,int r=V){
53     assert(-V<=l&&l<=r&&r<=V);
54     assert(-V<=x.k&&x.k<=V);
55     assert((__int128)x.k*l+x.b>=-inf&&(__int128)x.k*l+x.b<=inf);
56     assert((__int128)x.k*r+x.b>=-inf&&(__int128)x.k*r+x.b<=inf);
57     assert(cnt<INT_MAX-1);L.push_back(x);++cnt;
58     rt=upd(cnt,l,r,rt,-V,V);
59 }
60 ll ask(int x){
61     assert(-V<=x&&x<=V);
62     return ask(x,rt,-V,V);
63 }
64 };

```

2.4 ds/pq2stack.cpp

31 把优先队列删除转为栈撤销

什么时候用

适合只支持栈式加入/撤销，却需要删除最大权元素的离线或可撤销结构。模板只安排顺序，实际状态由调用方维护。

调用顺序与接口

- `pq2stack<T> q; q.set()` 初始化；`push(x,w)` 登记元素及删除优先级。
- 每次查询前先 `prep()`，按返回顺序把元素加入外部状态。
- `pop()` 删除最大权元素，返回外部栈需要撤销的次数；之后再次 `prep` 会给出重新加入的元素。

最小示例（放入 `main`）：

```

1 pq2stack<int> q; q.set(); vector<int> st;
2 q.push(10,1); q.push(20,2);
3 for(int x:q.prep())st.push_back(x);
4 int k=q.pop(); while(k--)st.pop_back();
5 for(int x:q.prep())st.push_back(x);
6 assert(st.size()==1 && st[0]==10);

```

使用边界

不能把 pop 的返回值当成被删元素。每次重加入都需要重新产生撤销记录；同权元素删除顺序不固定。单次可能重排大量元素，收益依赖整段操作序列。

完整代码

```

1  template<typename T>
2  struct pq2stack{
3  private:
4      struct info{
5          T e;int w;bool tag;
6          inline bool operator < (const info &x)const{return w<x.w;}
7      };
8      int n=0,r=0,d=0;
9      vector<info> I;
10 public:
11 void set(){n=r=d=0;I.clear();}
12 vector<T> prep(){
13     vector<T> ans;
14     while(d<I.size()) ans.push_back(I[d++].e);
15     return ans;
16 }
17 void push(T x,int w){
18     I.push_back({x,w,0});
19 }
20 void imerge(int i,int j,int k){
21     if(i==j||j==k) return;
22     static vector<info> y;
23     y.resize(k-i);
24     merge(I.begin()+i,I.begin()+j,I.begin()+j,I.begin()+k,y.begin());
25     for(int t=i;t<k;t++) I[t]=y[t-i];
26 }
27 int pop(){
28     // pq2stack core
29     assert(!I.empty());
30     int tar=(r?r&-r:inf<int>),mn=I.size()-1,hd=mn;
31     for(int i=I.size()-1;i>=0;i--){
32         if(I[i].tag){
33             I[i].tag=false;
34             tar--;
35         }
36         if(I[i]<I[mn]) mn=i;
37         else hd=i;
38         if(tar==0) break;
39     }
40     sort(I.begin()+n,I.end());
41     int cut=I.size();
42     for(int i=I.size()-1;i>hd;i--) if(I[i]<I[i-1]){
43         imerge(i,cut,I.size());
44         cut=i;
45     }
46     imerge(hd,cut,I.size());
47     int ans=max(0,d-hd);
48     int mx=(int)I.size()-1;
49     for(int i=n;i<(int)I.size();i++) if(I[i].w>I[mx].w) mx=i;
50     if(mx!=(int)I.size()-1) swap(I[mx],I.back());

```

```

51  I.pop_back();
52  chkmin(d,hd);
53  n=I.size();
54  if(r==0){
55      for(int i=0;i<n;i++) I[i].tag=true;
56      r=n;
57  }else{
58      for(int i=n-(r&-r)+1;i<n;i++) I[i].tag=true;
59      r--;
60  }
61  return ans;
62 }
63 };
64 // !!!!!!!!!!!!!!! before each query, use pq.prep() !!!!!!!!!!!!!!!

```

2.5 ds/xihe_tree.cpp

32 析合树：返回完整结构

什么时候用

对 $1..n$ 的排列构建析合树。构建时间 $O(n \log n)$ ，包含四毛子 RMQ、辅助线段树和返回结构在内的空间均为 $O(n)$ 。

调用顺序与接口

先粘贴 basic/ST table (linear).cpp。xihe_tree tr; auto t=tr.build(a) 接受不带哨兵的 vector；也可先 setN(n)，填写 tr.a[1..n]，再调用 build()。结果独立持有数据：rt 为根，cnt 为结点数，son[u] 为按位置排序的儿子；typ[u] 为 LEAF/INC/DEC/PRIME；[L[u],R[u]] 为下标闭区间，[mn[u],mx[u]] 为值域闭区间；id[i] 为第 i 项对应的叶子编号。

最小示例（放入 main）：

```

1 xihe_tree tr; auto t=tr.build(vector<int>{3,1,2});
2 int u=t.rt;
3 assert(t.L[u]==1 && t.R[u]==3);
4 assert(t.mn[u]==1 && t.mx[u]==3);
5 assert(t.typ[u]==xihe_tree::DEC && t.son[u].size()==2);
6 assert(t.mn[t.id[2]]==1);

```

使用边界

输入必须恰为 $1..n$ 的排列，不接受重复值；必要时在外部离散化，但需确认题目是否只关心排名连续。返回的结点编号、下标区间、id 均从 1 开始，0 不使用。 $n=0$ 时 $rt=cnt=0$ 。INC/DEC 是儿子值域顺序递增/递减的合点，PRIME 是析点。不要假定父编号大于儿子编号；在返回结果上自行遍历、DP，构建器不含 solve。再次 build 或修改构建器不会改变已返回结果。

完整代码

```

1 struct xihe_tree{
2  enum Type{LEAF,INC,DEC,PRIME};
3  struct result{
4      int n=0,rt=0,cnt=0;
5      vector<vector<int>> son;
6      vector<Type> typ;
7      vector<int> L,R,mn,mx,id;
8  };
9  int n=0;vector<int> a;
10 void set(){n=0;a.clear();}

```

```

11 void setN(int _n){assert(0<=_n&&_n<=INT_MAX/4);n=_n;a.assign(n+1,0);}
12 private:
13 struct segt{
14     int n;vector<int> val,laz;
15     segt(int _n):n(_n),val(4*n),laz(4*n){}
16     void push(int id,int x){val[id]+=x;laz[id]+=x;}
17     void pushdown(int id){if(laz[id]){push(id<<1,laz[id]);push(id<<1|1,
18         laz[id]);laz[id]=0;}}
19     void add(int ql,int qr,int x,int id,int l,int r){
20         if(ql<=l&&r<=qr){push(id,x);return;}
21         int mid=(l+r)>>1;pushdown(id);
22         if(ql<=mid)add(ql,qr,x,id<<1,l,mid);
23         if(mid<qr)add(ql,qr,x,id<<1|1,mid+1,r);
24         val[id]=min(val[id<<1],val[id<<1|1]);
25     }
26     void add(int l,int r,int x){add(l,r,x,1,1,n);}
27     int first(){
28         int id=1,l=1,r=n;assert(val[1]<=0);
29         while(l<r){
30             int mid=(l+r)>>1;pushdown(id);
31             if(val[id<<1]<=0){id<=1;r=mid;}
32             else{id=id<<1|1;l=mid+1;}
33         }
34         return l;
35     }
36 };
37 public:
38 result build()const{
39     assert(a.size()==(size_t)n+1||!n);
40     result res;res.n=n;
41     auto &son=res.son;auto &typ=res.typ;auto &L=res.L;auto &R=res.R;auto
42         &id=res.id;
43     int &cnt=res.cnt;
44     son.resize(2*n+1);typ.resize(2*n+1,LEAF);L.resize(2*n+1);R.resize(2*n+1);
45     id.resize(n+1);
46     if(!n){res.mn.resize(1);res.mx.resize(1);return res;}
47     vector<char> vis(n+1);
48     for(int i=1;i<=n;i++){assert(1<=a[i]&&a[i]<=n&&!vis[a[i]]);vis[a[i]]=true;}
49     vector<int> v(a.begin()+1,a.end());
50     Linear_RMQ<int> mn;Linear_RMQ<int,true> mx;mn.build(v);mx.build(v);
51     auto judge=[&](int l,int r){return mx.ask(l-1,r-1)-mn.ask(l-1,r-1)==r-l;};
52     vector<int> M(2*n+1),st1(n+1),st2(n+1),st(n+1);
53     int tp1=0,tp2=0,tp=0;segt T(n);
54     for(int i=1;i<=n;i++){
55         while(tp1&&a[i]<=a[st1[tp1]]) T.add(st1[tp1-1]+1,st1[tp1],
56             a[st1[tp1]]),tp1--;
57         while(tp2&&a[i]>=a[st2[tp2]]) T.add(st2[tp2-1]+1,st2[tp2],
58             -a[st2[tp2]]),tp2--;
59         T.add(st1[tp1]+1,i,-a[i]);
60         T.add(st2[tp2]+1,i,a[i]);
61         st1[++tp1]=st2[++tp2]=i;
62         id[i]=++cnt;L[cnt]=R[cnt]=M[cnt]=i;typ[cnt]=LEAF;
63         int le=T.first(),now=cnt;
64         while(tp&&L[st[tp]]>=le){
65             if((typ[st[tp]]==INC||typ[st[tp]]==DEC)&&judge(M[st[tp]],i)){

```

```

66         // 当儿子
67         R[st[tp]]=i;M[st[tp]]=L[now];son[st[tp]].push_back(now);
68         now=st[tp--];
69     }else if(judge(L[st[tp]],i)){
70         // 新合点
71         ++cnt;
72         if(a[L[st[tp]]]<a[i]) typ[cnt]=INC;
73         else typ[cnt]=DEC;
74         L[cnt]=L[st[tp]];R[cnt]=i;M[cnt]=L[now];
75         son[cnt].push_back(st[tp]);
76         son[cnt].push_back(now);
77         now=cnt;tp--;
78     }else{
79         // 新析点
80         ++cnt;typ[cnt]=PRIME;
81         son[cnt].push_back(now);
82         do son[cnt].push_back(st[tp--]);
83         while(tp&&!judge(L[st[tp]],i));
84         L[cnt]=(tp?L[st[tp]]:L[now]);R[cnt]=i;
85         if(tp) son[cnt].push_back(st[tp--]);
86         reverse(son[cnt].begin(),son[cnt].end());
87         now=cnt;
88     }
89 }
90 st[++tp]=now;
91 T.add(1,i,-1);
92 }
93 assert(tp==1);res.rt=st[1];
94 son.resize(cnt+1);typ.resize(cnt+1);L.resize(cnt+1);R.resize(cnt+1);
95 res.mn.resize(cnt+1);res.mx.resize(cnt+1);
96 for(int i=1;i<=cnt;i++){res.mn[i]=mn.ask(L[i]-1,R[i]-1);
97     res.mx[i]=mx.ask(L[i]-1,R[i]-1);}
98 return res;
99 }
100 result build(const vector<int> &v){
101     assert(v.size()<=(size_t)INT_MAX/4);setN(v.size());copy(v.begin(),v.end(),
102         a.begin()+1);return build();
103 }
104 };

```

3 图论

3.1 graph/flow.cpp

33 Dinic 最大流

什么时候用

有向网络最大流，容量是非负 ll 。一般图最坏 $O(V^2 E)$ ，具体模型常有更好表现。

调用顺序与接口

- `Flow_Graph g; g.setN(n); g.setST(s,t)` 初始化，点编号 $1..n$ ，源汇必须不同。
- `id=g.add_edge(u,v,cap)` 只添加一条有向边，自动生成反向残量边。
- `max_flow()` 返回累计最大流值；`edge_flow(id)` 查看返回的正向边上的流量。

最小示例（放入 main）：

```
1 Flow_Graph g; g.setN(3); g.setST(1,3);
2 int id=g.add_edge(1,2,5); g.add_edge(2,3,3);
3 assert(g.max_flow()==3 && g.edge_flow(id)==3);
4 assert(g.max_flow()==3);
```

使用边界

重复求流返回总流量，不是新增流量；添加边后继续增广。`setN` 清空图，`new_node` 保留旧图添加点。无向容量边需要根据建模显式加边。流量总和应在 ll 范围内。

完整代码

```
1 struct Flow_Graph{
2 private:
3 static const ll inf=2.02e18;
4 struct edge{int v,nxt;ll w;};
5 vector<edge> e=vector<edge>(2);bool is_flow=0;
6 int n=0,m=1,S=0,T=0;ll ans=0;
7 vector<int> lst{0},level{0},cur{0};
8 void add(int u,int v,ll w){
9     assert(m<INT_MAX-1);e.push_back({v,lst[u],w});lst[u]=++m;
10 }
11 bool BFS(){
12     for(int i=0;i<=n;i++) level[i]=-1;
13     queue<int> q;q.push(S);level[S]=0;
14     while(!q.empty()){
15         int u=q.front();q.pop();
16         if(u==T) return true;
17         for(int id=lst[u];id;id=e[id].nxt) if(e[id].w>0&&level[e[id].v]==-1){
18             level[e[id].v]=level[u]+1;
19             q.push(e[id].v);
20         }
21     }
22     return false;
23 }
24 ll DFS(int u,ll flow){
25     if(u==S) return flow;
26     ll now=0;
27     for(int &i=cur[u];i;i=e[i].nxt) if(level[e[i].v]==level[u]-1&&e[i].w){
28         ll fo=DFS(e[i].v,min(e[i].w,flow-now));
29         e[i].w-=fo;e[i].w+=fo;now+=fo;
```

```

30     if(now==flow) break;
31 }
32 return now;
33 }
34 public:
35 void set(){
36     n=S=T=0;m=1;ans=0;is_flowед=false;e.assign(2,{0,0,0});
37     lst.assign(1,0);level.assign(1,0);cur.assign(1,0);
38 }
39 void setN(int _n){
40     assert(0<=_n&&_n<INT_MAX);set();n=_n;
41     lst.assign(n+1,0);level.assign(n+1,0);cur.assign(n+1,0);
42 }
43 void setST(int _S,int _T){
44     assert(1<=min(_S,_T)&&max(_S,_T)<INT_MAX&&_S!=_T);
45     if(S!=_S||T!=_T){
46         for(int i=2;i<=m;i+=2)e[i].w+=e[i^1].w,e[i^1].w=0;
47         ans=0;
48     }
49     S=_S;T=_T;n=max({n,S,T});
50     lst.resize(n+1);level.resize(n+1);cur.resize(n+1);is_flowед=false;
51 }
52 int new_node(){
53     assert(n<INT_MAX-1);++n;lst.push_back(0);level.push_back(0);
54     cur.push_back(0);
55     is_flowед=false;return n;
56 }
57 ll edge_flow(int id){
58     assert(2<=id&&id<=m&&id%2==0);
59     return e[id^1].w;
60 }
61 int add_edge(int u,int v,ll w){
62     // return forward edge id
63     assert(1<=min(u,v)&&max(u,v)<=n&&0<=w);
64     assert(m&1);
65     is_flowед=false;add(u,v,w);add(v,u,0);return m-1;
66 }
67 ll max_flow(){
68     assert(1<=min(S,T)&&max(S,T)<=n&&S!=T);
69     if(is_flowед) return ans;
70     is_flowед=true;
71     while(BFS()){
72         for(int i=1;i<=n;i++) cur[i]=lst[i];
73         ll f=DFS(T,inf);assert(ans<=LLONG_MAX-f);ans+=f;
74     }
75     return ans;
76 }
77 };

```

3.2 graph/mcmf.cpp

34 最小费用最大流

什么时候用

先求最大可行流量，再在最大流中最小化费用。有负费用边时先用 SPFA 求势能；之后 Dijkstra 求最短路，在零约化费用子图上用 Dinic 批量增广，适合大量路径费用相同的网络。DFS 使用显式数组，不依赖递归栈。

调用顺序与接口

- `Cost_Flow_Graph g; g.setN(n); g.setST(s,t)` 初始化。
- `add_edge(u,v,cap,cost)` 添加有向边并返回正向边编号。
- `min_cost_flow()` 返回 {总流量, 总费用}; `edge_flow(id)` 查询流量。

最小示例（放入 main）:

```
1 Cost_Flow_Graph g; g.setN(3); g.setST(1,3);
2 int id=g.add_edge(1,2,2,-1); g.add_edge(2,3,2,3);
3 auto [f,c]=g.min_cost_flow();
4 assert(f==2 && c==4 && g.edge_flow(id)==2);
```

使用边界

允许负费用边，但整个输入图不能有负费用环。没有指定流量参数；要限制流量，增加一个容量为上限的新源边。更改图后重新从原始容量计算；不改图则返回缓存。容量、总流量及计算过程中的总费用须能用 `ll` 表示，距离与势能使用 `__int128`。相同最短路费用的增广会合并，但费用层数很多时仍可能较慢。

完整代码

```
1 struct Cost_Flow_Graph{
2 private:
3 static constexpr __int128 inf=(__int128)1<<120;
4 struct edge{int v,nxt;ll w,c;};
5 vector<edge> e=vector<edge>(2);bool is_flowd=false;
6 int n=0,m=1,S=0,T=0;ll ans=0,cost=0;
7 vector<int> lst{0},pre{0},dep,cur,q;
8 vector<ll> lim,rem;
9 vector<__int128> h{0},dis{0};
10 void add(int u,int v,ll w,ll c){
11     assert(m<INT_MAX-1);e.push_back({v,lst[u],w,c});lst[u]=++m;
12 }
13 void SPFA(){
14     fill(h.begin(),h.end(),0);
15     bool neg=false;
16     for(int i=2;i<=m;i++)if(e[i].w&&e[i].c<0){neg=true;break;}
17     if(!neg)return;
18     vector<int> cnt(n+1);vector<char> inq(n+1,true);
19     queue<int> q;for(int i=1;i<=n;i++)q.push(i);
20     while(!q.empty()){
21         int u=q.front();q.pop();inq[u]=false;
22         for(int id=lst[u];id;id=e[id].nxt)if(e[id].w){
23             int v=e[id].v;
24             if(h[v]<=h[u]+e[id].c)continue;
25             h[v]=h[u]+e[id].c;cnt[v]=cnt[u]+1;
26             assert(cnt[v]<n);
27             if(!inq[v])inq[v]=true,q.push(v);
28         }
29     }
```

```

30 }
31 bool Dijkstra(){
32     fill(dis.begin(),dis.end(),inf);dis[S]=0;
33     priority_queue<pair<__int128,int>,vector<pair<__int128,int>>>,
34         greater<pair<__int128,int>>> q;
35     q.push({0,S});
36     while(!q.empty()){
37         auto [d,u]=q.top();q.pop();
38         if(d!=dis[u])continue;
39         if(u==T)break;
40         for(int id=lst[u];id;id=e[id].nxt)if(e[id].w){
41             int v=e[id].v;__int128 w=e[id].c+h[u]-h[v];
42             assert(w>=0);
43             if(dis[v]<=d+w)continue;
44             dis[v]=d+w;q.push({dis[v],v});
45         }
46     }
47     if(dis[T]==inf)return false;
48     for(int i=1;i<=n;i++)h[i]+=min(dis[i],dis[T]);
49     return true;
50 }
51 bool BFS(){
52     fill(dep.begin(),dep.end(),-1);dep[S]=0;
53     int l=0,r=0;q[r++]=S;
54     while(l<r){
55         int u=q[l++];
56         for(int id=lst[u];id;id=e[id].nxt){
57             int v=e[id].v;
58             if(!e[id].w||dep[v]!=-1||h[v]!=h[u]+e[id].c)continue;
59             dep[v]=dep[u]+1;q[r++]=v;
60         }
61     }
62     return dep[T]!=-1;
63 }
64 ll DFS(){
65     // iterative DFS: rem[u] is the part of lim[u] not sent yet
66     int u=S;lim[u]=rem[u]=LLONG_MAX;
67     while(true){
68         if(u==T)rem[u]=0;
69         int &id=cur[u];
70         while(rem[u]&&id){
71             int v=e[id].v;
72             if(e[id].w&&dep[v]==dep[u]+1&&h[v]==h[u]+e[id].c)break;
73             id=e[id].nxt;
74         }
75         if(!rem[u]||!id){
76             ll f=lim[u]-rem[u];
77             if(u==S)return f;
78             if(rem[u])dep[u]=-1;
79             int x=pre[u];u=e[x^1].v;
80             e[x].w-=f;e[x^1].w+=f;rem[u]-=f;
81             if(rem[u])cur[u]=e[x].nxt;
82         }else{
83             int v=e[id].v;pre[v]=id;
84             lim[v]=rem[v]=min(rem[u],e[id].w);u=v;

```

```

85     }
86 }
87 }
88 public:
89 void set(){
90     n=S=T=0;m=1;ans=cost=0;is_flowd=false;e.assign(2,{0,0,0,0});
91     dep.clear();cur.clear();q.clear();lim.clear();rem.clear();
92     lst.assign(1,0);pre.assign(1,0);h.assign(1,0);dis.assign(1,0);
93 }
94 void setN(int _n){
95     assert(0<=_n&&_n<INT_MAX);set();n=_n;
96     lst.assign(n+1,0);pre.assign(n+1,0);h.assign(n+1,0);dis.assign(n+1,0);
97 }
98 void setST(int _S,int _T){
99     assert(1<=min(_S,_T)&&max(_S,_T)<INT_MAX&&_S!=_T);
100     S=_S;T=_T;n=max({n,S,T});
101     lst.resize(n+1);pre.resize(n+1);h.resize(n+1);dis.resize(n+1);
102     is_flowd=false;
103 }
104 int new_node(){
105     assert(n<INT_MAX-1);++n;lst.push_back(0);pre.push_back(0);h.push_back(0);
106     dis.push_back(0);
107     is_flowd=false;return n;
108 }
109 ll edge_flow(int id){
110     assert(2<=id&&id<=m&&id%2==0);
111     return e[id^1].w;
112 }
113 int add_edge(int u,int v,ll w,ll c){
114     // return forward edge id
115     assert(1<=min(u,v)&&max(u,v)<=n&&w>=0&&c!=LLONG_MIN);
116     assert(m&1);
117     is_flowd=false;add(u,v,w,c);add(v,u,0,-c);return m-1;
118 }
119 pair<ll,ll> min_cost_flow(){
120     assert(1<=min(S,T)&&max(S,T)<=n&&S!=T);
121     if(is_flowd)return {ans,cost};
122     for(int i=2;i<=m;i+=2)e[i].w+=e[i^1].w,e[i^1].w=0;
123     ans=cost=0;SPFA();
124     dep.resize(n+1);cur.resize(n+1);q.resize(n+1);
125     lim.resize(n+1);rem.resize(n+1);
126     while(Dijkstra())while(BFS()){
127         cur=lst;ll f=DFS();assert(f>0);
128         __int128 c=h[T]-h[S];
129         assert(ans<=LLONG_MAX-f);
130         assert(c>=((__int128)LLONG_MIN-cost)/f&&c<=((__int128)LLONG_MAX-cost)/f);
131         ans+=f;cost=(ll)(cost+c*f);
132     }
133     is_flowd=true;return {ans,cost};
134 }
135 };

```

3.3 graph/2-sat.cpp

36 布尔约束求解

什么时候用

求满足所有二元或子句的一组赋值，时间与空间 $O(n+m)$ 。

调用顺序与接口

- `two_sat s; s.setN(n)` 初始化 n 个布尔变量，下标 $1..n$ 。
- `add(x,a,y,b)` 表示 (变量 x 等于 a) 或 (变量 y 等于 b)， a,b 为 $0/1$ 。
- `solve()` 无解返回空 vector，否则 `ans[1..n]` 为可行赋值。

最小示例（放入 main）：

```
1 two_sat s; s.setN(2);
2 s.add(1,1,1,1); // force x1=true
3 s.add(1,0,2,1); // x1 implies x2
4 auto ans=s.solve();
5 assert(!ans.empty() && ans[1] && ans[2]);
```

使用边界

只有“或”子句接口；例如强制 $x=a$ 可重复同一文字。setN 清空约束。不要把 add 当作直接添加两条蕴含边。

完整代码

```
1 struct two_sat{
2     int tot=0,scnt=0,top=0,n=0;
3     vector<int> low,dfn,stk,bel;
4     vector<vector<int>> edg;vector<char> inq;
5     void set(){setN(0);}
6     void setN(int _n){
7         // 1-index, 2i-1 true 2i false
8         assert(0<=_n&&_n<(INT_MAX-2)/2);n=_n;
9         low.assign(2*n+2,0);dfn.assign(2*n+2,0);stk.assign(2*n+2,0);
10        bel.assign(2*n+2,0);inq.assign(2*n+2,0);edg.assign(2*n+2,{});
11        tot=scnt=top=0;
12    }
13    void tarjan(int u){
14        vector<pair<int,int>> q{{u,0}};
15        low[u]=dfn[u]=++tot;stk[++top]=u;inq[u]=true;
16        while(!q.empty()){
17            auto &[x,i]=q.back();
18            if(i<(int)edg[x].size()){
19                int v=edg[x][i++];
20                if(!dfn[v]){
21                    low[v]=dfn[v]=++tot;stk[++top]=v;inq[v]=true;q.push_back({v,0});
22                }else if(inq[v])low[x]=min(low[x],dfn[v]);
23            }else{
24                int v=x;q.pop_back();
25                if(!q.empty()){int p=q.back().first;low[p]=min(low[p],low[v]);}
26                if(low[v]==dfn[v]){
27                    scnt++;int y;
28                    do{y=stk[top--];inq[y]=false;bel[y]=scnt;}while(y!=v);
29                }
30            }
31        }
```

```

32 }
33 vector<int> solve(){
34     // if impossible return empty
35     // else return a vector with length n+1, the a[i] - any valid answer
36     tot=scnt=top=0;
37     for(int i=1;i<=2*n;i++) low[i]=dfn[i]=bel[i]=inq[i]=0;
38     for(int i=1;i<=2*n;i++) if(!dfn[i]) tarjan(i);
39     for(int i=1;i<=n;i++) if(bel[2*i-1]==bel[2*i]) return {};
40     vector<int> ans(n+1);
41     for(int i=1;i<=n;i++) ans[i]=bel[2*i-1]<bel[2*i];
42     return ans;
43 }
44 void add(int x1,int o1,int x2,int o2){
45     // x1 = o1 or x2 = o2
46     assert(1<=min(x1,x2)&&max(x1,x2)<=n&&0<=min(o1,o2)&&max(o1,o2)<=1);
47     edg[2*x1-(!o1)].push_back(2*x2-o2);
48     edg[2*x2-(!o2)].push_back(2*x1-o1);
49 }
50 };

```

3.4 graph/block-cut-tree.cpp

37 点双连通分量与圆方树

什么时候用

无向图的点双分解，支持非连通图和重边，不支持自环。构建时间 $O(n+m)$ 。

调用顺序与接口

- `block_cut g; g.setN(n)` 后 `add_edge`，再 `build`。
- 原点仍为 $1..n$ ，方点为 $n+1..scc$ ，`tr` 是圆方森林邻接表。
- 第 k 个方点 $n+k$ 的原图边存在 `ec[k]` 中。`ff`、`dep` 是森林上的父亲和深度。

最小示例（放入 main）：

```

1 block_cut g; g.setN(3);
2 g.add_edge(1,2); g.add_edge(2,3); g.build();
3 assert(g.scc==5 && g.tr[2].size()==2);

```

使用边界

`scc` 是最终最大节点编号，不是分量个数，方点数应为 `scc-n`。割点可以属于多个点双，不能像 SCC 一样给每个原点分配唯一块号。递归深度可能达到 $O(n)$ 。

完整代码

```

1 struct block_cut{
2     int n=0,tot=0,scc=0,top=0;
3     vector<char> inq;
4     vector<int> low,dfn,stk,dep,ff;
5     vector<vector<int>>> edg,tr;
6     vector<vector<array<int,2>>> ec;
7     void setN(int _n){
8         assert(0<=_n&&_n<(INT_MAX-2)/2);n=_n;
9         low.assign(n+2,0);dfn.assign(n+2,0);stk.assign(n+2,0);inq.assign(n+2,0);
10        edg.assign(n+2,{});tr.assign(2*n+2,{});ec.assign(n+2,{});

```

```

11  dep.assign(2*n+2,0);ff.assign(2*n+2,0);tot=scc=top=0;
12  }
13  void add_edge(int u,int v){
14      assert(1<=min(u,v)&&max(u,v)<=n&&u!=v);
15      edg[u].push_back(v);
16      edg[v].push_back(u);
17  }
18  void tarjan(int u,int fa=0){
19      // frame: vertex, parent, next edge, skipped parent edge
20      vector<array<int,4>> q{{u,fa,0,0}};
21      low[u]=dfn[u]=++tot;stk[++top]=u;inq[u]=true;
22      while(!q.empty()){
23          auto &[x,p,i,skip]=q.back();
24          if(i<(int)edg[x].size()){
25              int v=edg[x][i++];
26              if(v==p&&!skip){skip=1;continue;}
27              if(!dfn[v]){
28                  low[v]=dfn[v]=++tot;stk[++top]=v;inq[v]=true;q.push_back({v,x,
29                      0,0});
30              }else if(inq[v])low[x]=min(low[x],dfn[v]);
31          }else{
32              int v=x,parent=p;q.pop_back();
33              if(q.empty())continue;
34              low[parent]=min(low[parent],low[v]);
35              if(low[v]>=dfn[parent]){
36                  ++scc;tr[scc].push_back(parent);tr[parent].push_back(scc);int y;
37                  do{y=stk[top--];inq[y]=false;tr[y].push_back(scc);
38                      tr[scc].push_back(y);}while(y!=v);
39              }
40          }
41      }
42  }
43  void dfs(int u,int fa){
44      vector<int> q{u};ff[u]=fa;dep[u]=dep[fa]+1;
45      for(int i=0;i<(int)q.size();i++){
46          int x=q[i];
47          for(int v:tr[x])if(v!=ff[x]){ff[v]=x;dep[v]=dep[x]+1;q.push_back(v);}
48      }
49  }
50  void build(){
51      fill(dfn.begin(),dfn.end(),0);fill(inq.begin(),inq.end(),0);
52      fill(dep.begin(),dep.end(),0);fill(ff.begin(),ff.end(),0);
53      for(auto &v:tr)v.clear();for(auto &v:ec)v.clear();
54      tot=top=0;scc=n;
55      for(int i=1;i<=n;i++)if(!dfn[i]){
56          tarjan(i);dfs(i,0);inq[i]=false;top=0;
57      }
58      for(int i=1;i<=n;i++)for(int j:edg[i])if(i<j){
59          int x=ff[i]==ff[j]?ff[i]:(dep[i]>dep[j]?ff[i]:ff[j]);
60          assert(n<x&&x<=scc);ec[x-n].push_back({i,j});
61      }
62  }
63  };

```

3.5 graph/blossom.cpp

39 无权一般图最大匹配

什么时候用

带花树，允许奇环，返回匹配边数，最坏 $O(n^3)$ 。

调用顺序与接口

- blossom g; g.setN(n) 后 add_edge 添加无向边。
- solve() 返回最大匹配的边数；match[u] 为匹配点，未匹配为 0。
- solve()*2==n 判定存在完全匹配。

最小示例（放入 main）：

```
1 blossom g; g.setN(4);
2 g.add_edge(1,2); g.add_edge(2,3);
3 g.add_edge(3,1); g.add_edge(3,4);
4 assert(g.solve()==2);
5 assert(g.match[g.match[1]]==1);
```

使用边界

这是无权匹配，不会最小化或最大化边权。点数为奇数时不可能完全匹配。重复 solve 会重新求解。

完整代码

```
1 struct blossom{
2     int n=0;
3     vector<vector<int>> edg;
4     vector<int> match,pre,bel,q;
5     vector<char> inq,vis,flower;
6     void setN(int _n){
7         assert(0<=_n&&_n<INT_MAX);n=_n;
8         edg.assign(n+1,{});match.assign(n+1,0);
9         pre.resize(n+1);bel.resize(n+1);q.resize(n+1);
10        inq.resize(n+1);vis.resize(n+1);flower.resize(n+1);
11    }
12    void add_edge(int u,int v){
13        assert(1<=min(u,v)&&max(u,v)<=n);
14        if(u==v)return;
15        edg[u].push_back(v);edg[v].push_back(u);
16    }
17    int lca(int u,int v){
18        fill(vis.begin(),vis.end(),false);
19        while(1){
20            u=bel[u];vis[u]=true;
21            if(!match[u])break;
22            u=pre[match[u]];
23        }
24        while(!vis[bel[v]])v=pre[match[bel[v]]];
25        return bel[v];
26    }
27    void shrink(int u,int v,int w){
28        while(bel[u]!=w){
29            flower[bel[u]]=flower[bel[match[u]]]=true;
30            pre[u]=v;v=match[u];u=pre[v];
31        }
```

```

32 }
33 bool BFS(int s){
34     fill(inq.begin(),inq.end(),false);
35     fill(pre.begin(),pre.end(),0);
36     iota-bel.begin(),bel.end(),0);
37     int h=0,t=0;q[t++]=s;inq[s]=true;
38     while(h<t){
39         int u=q[h++];
40         for(int v:edg[u]){
41             if-bel[u]==bel[v]||match[u]==v)continue;
42             if(v==s||(match[v]&&pre[match[v]])){
43                 int w=lca(u,v);
44                 fill(flower.begin(),flower.end(),false);
45                 shrink(u,v,w);shrink(v,u,w);
46                 for(int i=1;i<=n;i++)if(flower[bel[i]]){
47                     bel[i]=w;
48                     if(!inq[i])inq[i]=true,q[t++]=i;
49                 }
50             }else if(!pre[v]){
51                 pre[v]=u;
52                 if(!match[v]){
53                     while(v){
54                         int x=pre[v],y=match[x];
55                         match[v]=x;match[x]=v;v=y;
56                     }
57                     return true;
58                 }
59                 int x=match[v];inq[x]=true;q[t++]=x;
60             }
61         }
62     }
63     return false;
64 }
65 int solve(){
66     fill(match.begin(),match.end(),0);
67     int ans=0;
68     for(int i=1;i<=n;i++)if(!match[i])ans+=BFS(i);
69     return ans;
70 }
71 };

```

3.6 graph/bipartite-matching.cpp

40 二分图最大匹配

什么时候用

给定左右两部，多次加边后求最大基数匹配，返回匹配的点对。先贪心匹配，再用多源 BFS 记录起点与前驱，遇到未匹配右点就立即增广；整个过程不递归。

调用顺序与接口

- `Bipartite_Matching g; g.setN(n,m)`，左部下标 $1..n$ ，右部下标 $1..m$ ；两部编号互不影响。
- `g.add_edge(u,v)` 加入左点 u 到右点 v 的边；允许重边。
- `auto a=g.solve()` 返回 `vector<pair<int,int>>`，每个二元组 (u,v) 是一对匹配的左点和右点，按左点递增；

`a.size()` 就是最大匹配数。

- 求解后的 `matchL[u]`、`matchR[v]` 是对应另一部的点，未匹配为 0。`set()` 清空，`setN` 清空并设置两部大小。

最小示例（放入 `main`）：

```
1 Bipartite_Matching g; g.setN(3,3);
2 g.add_edge(1,1); g.add_edge(2,1);
3 g.add_edge(2,2); g.add_edge(3,2);
4 auto a=g.solve(); assert(a.size()==2);
5 g.add_edge(3,3); a=g.solve(); assert(a.size()==3);
6 for(auto [u,v]:a){
7     assert(g.matchL[u]==v && g.matchR[v]==u);
8 }
```

使用边界

邻接表在 `solve` 内按左点压成连续存储，保持各点的加边顺序。`rt[u]` 记录搜索树的未匹配起点，`pre[u]` 记录增广前驱；某起点完成增广后，跳过该搜索树剩余队列项。一轮允许处理不同长度的增广路。设总点数 $V=n+m$ 、边数 E ，空间 $O(V+E)$ ，每轮 $O(n+E)$ ；保守最坏时间界为 $O((V+E)(\min(n,m)+1))$ ，不沿用分层 HK 的复杂度保证。空部、空边集允许，返回空匹配。每次 `solve` 都重新计算；可继续加边再调用。

完整代码

```
1 struct Bipartite_Matching{
2     struct edge{int u,v;};
3     int n=0,m=0;
4     vector<edge> e;
5     vector<int> matchL,matchR;
6     void set(){n=m=0;e.clear();matchL.clear();matchR.clear();}
7     void setN(int _n,int _m){
8         assert(0<=_n&&_n<INT_MAX-1&&0<=_m&&_m<INT_MAX);set();n=_n;m=_m;
9         matchL.assign(n+1,0);matchR.assign(m+1,0);
10    }
11    void add_edge(int u,int v){
12        assert(1<=u&&u<=n&&1<=v&&v<=m&&e.size()<INT_MAX);
13        e.push_back({u,v});
14    }
15    vector<pair<int,int>> solve(){
16        matchL.assign(n+1,0);matchR.assign(m+1,0);
17        vector<int> head(n+2),to(e.size()),rt(n+1),pre(n+1),q;
18        for(auto [u,v]:e)head[u+1]++;
19        partial_sum(head.begin(),head.end(),head.begin());
20        copy(head.begin(),head.begin()+n+1,rt.begin());
21        for(auto [u,v]:e)to[rt[u]++]=v;
22        q.reserve(n);
23        for(int u=1;u<=n;u++)for(int i=head[u];i<head[u+1];i++)if(!matchR[to[i]]){
24            matchL[u]=to[i];matchR[to[i]]=u;break;
25        }
26        while(true){
27            fill(rt.begin(),rt.end(),0);q.clear();bool found=false;
28            for(int u=1;u<=n;u++)if(!matchL[u]&&head[u]!=head[u+1]){
29                rt[u]=u;pre[u]=0;q.push_back(u);
30            }
31            for(int h=0;h<(int)q.size();h++){
32                int u=q[h];if(matchL[rt[u]])continue;
33                for(int i=head[u];i<head[u+1];i++){
34                    int v=to[i],w=matchR[v];
```

```

35         if(!w){
36             found=true;
37             for(int x=u;v;x=pre[x]){
38                 int t=matchL[x];matchL[x]=v;matchR[v]=x;v=t;
39             }
40             break;
41         }
42         if(!rt[w])rt[w]=rt[u],pre[w]=u,q.push_back(w);
43     }
44 }
45 if(!found)break;
46 }
47 vector<pair<int,int>> ans;ans.reserve(min(n,m));
48 for(int u=1;u<=n;u++)if(matchL[u])ans.emplace_back(u,matchL[u]);
49 return ans;
50 }
51 };

```

3.7 graph/directed-mst.cpp

42 指定根的最小树形图

什么时候用

在带权有向图中，以 S 为根，选取 $n-1$ 条边，使 S 能沿选边到达每个点，且总权最小。每个非根点恰有一条入边。

调用顺序与接口

- `Directed_MST g; g.setN(n); id=g.add_edge(u,v,w)` 加入 u 指向 v 的边， w 为 ll ，返回从 1 开始的边编号。
- `auto a=g.solve(S)` 返回 `result`。`a.ok=false` 表示不存在以 S 为根的生成树形图，此时 `id` 为空。
- 有解时 `a.cost` 为总权，`a.id[v]` 为 v 的入边编号，`a.id[S]=0`；遍历所有非根点即可得到实际边集，父亲是 `g.e[a.id[v]].u`。
- 点和 `id` 数组均为 1 下标。 $n=1$ 时有解、总权为 0；要求 $1 \leq S \leq n$ 。允许负权、重边、自环；同端点只保留最小权，平权保留较早加入的边。

最小示例（放入 main）：

```

1 Directed_MST g; g.setN(4);
2 g.add_edge(1,2,5); g.add_edge(1,3,8);
3 g.add_edge(2,3,-2); g.add_edge(3,2,-1);
4 g.add_edge(3,4,1);
5 auto a=g.solve(1);
6 assert(a.ok && a.cost==4 && a.id[1]==0);
7 assert(a.id[2]==1 && a.id[3]==3 && a.id[4]==5);
8 assert(!g.solve(4).ok);

```

使用边界

设 n 为点数、 m 为输入边数，时间 $O((n+m)\log(n+1))$ ，空间 $O(n+m)$ ；任意显式图至少需要 $O(m)$ 读边，只有稀疏图才能写成 n 乘多对数。使用左偏堆、可回退并查集缩环并恢复选边。中间权值和 `cost` 都是 `__int128`；输出总权时先抄 `basic/int128_IO.cpp`，不要直接强转 `ll`。`solve` 不修改原图，可更换根重新求解。

完整代码

```

1 struct Directed_MST{
2 struct edge{int u,v;ll w;};
3 struct result{bool ok=false;__int128 cost=0;vector<int> id;};
4 int n=0;
5 vector<edge> e=vector<edge>(1);
6 void set(){n=0;e.assign(1,{})};
7 void setN(int _n){assert(0<=_n&&_n<INT_MAX);set();n=_n;}
8 int add_edge(int u,int v,ll w){
9     assert(1<=min(u,v)&&max(u,v)<=n&&e.size()<INT_MAX);
10    int id=e.size();e.push_back({u,v,w});return id;
11 }
12 private:
13 struct heap{
14     struct node{int ls=0,rs=0,dis=0,id=0;__int128 w=0,lazy=0;};
15     vector<node> a=vector<node>(1);
16     void add(int u,__int128 w){if(u)a[u].w+=w,a[u].lazy+=w;}
17     void down(int u){add(a[u].ls,a[u].lazy);add(a[u].rs,a[u].lazy);a[u].lazy=0;}
18     int merge(int u,int v){
19         if(!u||!v)return u|v;
20         if(a[u].w>a[v].w||(a[u].w==a[v].w&&a[u].id>a[v].id))swap(u,v);
21         down(u);a[u].rs=merge(a[u].rs,v);
22         if(a[a[u].ls].dis<a[a[u].rs].dis)swap(a[u].ls,a[u].rs);
23         a[u].dis=a[a[u].rs].dis+1;return u;
24     }
25     int push(int u,int id,ll w){
26         a.push_back({0,0,1,id,w,0});return merge(u,(int)a.size()-1);
27     }
28     int pop(int u){down(u);return merge(a[u].ls,a[u].rs);}
29 };
30 struct DSU{
31     vector<int> fa,siz;
32     vector<pair<int,int>> stk;
33     DSU(int n):fa(n+1),siz(n+1,1){iota(fa.begin(),fa.end(),0);}
34     int find(int u)const{while(fa[u]!=u)u=fa[u];return u;}
35     bool join(int u,int v){
36         u=find(u);v=find(v);if(u==v)return false;
37         if(siz[u]<siz[v])swap(u,v);
38         stk.emplace_back(u,v);fa[v]=u;siz[u]+=siz[v];return true;
39     }
40     void rollback(int t){
41         while((int)stk.size()>t){auto [u,v]=stk.back();stk.pop_back();fa[v]=v;
42             siz[u]-=siz[v];}
43     }
44 };
45 public:
46 result solve(int S)const{
47     assert(1<=S&&S<=n);DSU dsu(n);heap h;
48     vector<int> rt(n+1),seen(n+1),path(n),q(n),in(n+1);
49     // Keep one cheapest edge per ordered pair, also bounding heap size by n*n.
50     {
51         vector<vector<int>> edg(n+1);vector<int> vis(n+1),best(n+1),a;
52         for(int id=1;id<(int)e.size();
53             id++)if(e[id].u!=e[id].v&&e[id].v!=S)edg[e[id].v].push_back(id);
54         for(int v=1;v<=n;v++){

```

```

55     a.clear();
56     for(int id:edg[v]){
57         int u=e[id].u;
58         if(vis[u]!=v)vis[u]=v,best[u]=id,a.push_back(u);
59         else if(e[id].w<e[best[u]].w)best[u]=id;
60     }
61     for(int u:a)rt[v]=h.push(rt[v],best[u],e[best[u]].w);
62 }
63 }
64 struct cycle{int u,t;vector<int> e;};
65 vector<cycle> cyc;__int128 cost=0;seen[S]=-1;
66 for(int s=1;s<=n;s++){
67     int u=dsu.find(s),top=0;
68     while(!seen[u]){
69         while(rt[u]&&dsu.find(e[h.a[rt[u]].id].u)==u)rt[u]=h.pop(rt[u]);
70         if(!rt[u])return {};
71         int id=h.a[rt[u]].id;__int128 w=h.a[rt[u]].w;
72         cost+=w;h.add(rt[u],-w);rt[u]=h.pop(rt[u]);
73         q[top]=id;path[top++]=u;seen[u]=s;u=dsu.find(e[id].u);
74         if(seen[u]==s){
75             int end=top,t=dsu.stk.size(),r=0,v;
76             do{v=path[--top];r=h.merge(r,rt[v]);}while(dsu.join(u,v));
77             u=dsu.find(u);rt[u]=r;seen[u]=0;
78             cyc.push_back({u,t,vector<int>(q.begin()+top,q.begin()+end)});
79         }
80     }
81     for(int i=0;i<top;i++)in[dsu.find(e[q[i]].v)]=q[i];
82 }
83 for(int i=(int)cyc.size()-1;i>=0;i--){
84     auto &c=cyc[i];int id=in[c.u];assert(id);
85     dsu.rollback(c.t);
86     for(int x:c.e)in[dsu.find(e[x].v)]=x;
87     in[dsu.find(e[id].v)]=id;
88 }
89 return {true,cost,move(in)};
90 }
91 };

```

3.8 graph/chordal.cpp

43 弦图判定与完美消除序列

什么时候用

判断无向图中是否不存在长度至少为 4 的无弦诱导环。如果是弦图，返回一个完美消除序列。

调用顺序与接口

- Chordal g; g.setN(n)，点下标 1..n; g.add_edge(u,v) 加无向边。自环忽略，重边在求解时去重。
- bool ok=g.solve()。成功时 peo[0..n-1] 按从前到后给出消除顺序，rk[u] 是 u 在序列中的 1 下标位置。
- 依次消除 peo 中的点时，每个点尚未消除的邻居构成团。不要把 MCS 的选点顺序直接作为消除顺序，两者相反。
- 失败时清空 peo、rk。空图和非连通图均支持；可以继续加边再求解。

最小示例（放入 main）：

```

1 Chordal g; g.setN(4);
2 g.add_edge(1,2); g.add_edge(2,3);
3 g.add_edge(3,4); g.add_edge(4,1);
4 assert(!g.solve() && g.peo.empty());
5 g.add_edge(1,3); assert(g.solve());
6 assert(g.peo.size()==4);
7 for(int i=0;i<4;i++)assert(g.rk[g.peo[i]]==i+1);

```

使用边界

时间、空间均为 $O(n+m)$, m 为输入边数。MCS 用桶维护已选邻居数；验证时把结点按最靠前的后继邻居分组，统一检查后继邻居的包含关系，避免对每个邻域两两枚举。返回 false 时只给出判定，不构造无弦环证据。

完整代码

```

1 struct Chordal{
2 int n=0;
3 vector<vector<int>> edg;
4 vector<int> peo,rk;
5 void set(){n=0;edg.clear();peo.clear();rk.clear();}
6 void setN(int _n){assert(0<=_n&&_n<INT_MAX);set();n=_n;edg.resize(n+1);}
7 void add_edge(int u,int v){
8     assert(1<=min(u,v)&&max(u,v)<=n);
9     if(u!=v)edg[u].push_back(v),edg[v].push_back(u);
10 }
11 bool solve(){
12     vector<int> vis(n+1),cnt(n+1);vector<vector<int>> bucket(n+1),son(n+1);
13     for(int u=1;u<=n;u++){
14         int k=0;
15         for(int v:edg[u])if(vis[v]!=u)vis[v]=u,edg[u][k++]=v;
16         edg[u].resize(k);bucket[0].push_back(u);
17     }
18     peo.resize(n);rk.assign(n+1,0);int mx=0;
19     for(int i=n;i>=1;i--){
20         while(true){
21             while(bucket[mx].empty())mx--;
22             int u=bucket[mx].back();bucket[mx].pop_back();
23             if(rk[u]||cnt[u]!=mx)continue;
24             peo[i-1]=u;rk[u]=i;
25             for(int v:edg[u])if(!rk[v]){
26                 bucket[++cnt[v]].push_back(v);mx=max(mx,cnt[v]);
27             }
28             break;
29         }
30     }
31     for(int u=1;u<=n;u++){
32         int p=0;
33         for(int v:edg[u])if(rk[v]>rk[u]&&(!p||rk[v]<rk[p]))p=v;
34         if(p)son[p].push_back(u);
35     }
36     fill(vis.begin(),vis.end(),0);
37     for(int p=1;p<=n;p++){
38         for(int v:edg[p])vis[v]=p;
39         for(int u:son[p])for(int v:edg[u])if(v!=p&&rk[v]>rk[u]&&vis[v]!=p){
40             peo.clear();rk.clear();return false;
41         }
42     }

```

```
43     return true;
44 }
45 };
```

4 树上算法

4.1 trees/cartesian.cpp

46 小根堆笛卡尔树

什么时候用

$O(n)$ 构建，同时满足下标的中序顺序与值的小根堆序，空间 $O(n)$ 。

调用顺序与接口

`auto [ls,rs]=cartesian(a)` 接受从 0 开始的 vector，返回两个同长度 vector，分别为左右儿子下标。无儿子用 -1。相等值保留较早下标在上方。根是原数组中第一个最小值的位置。

最小示例（放入 main）：

```
1 vector<int> a{3,1,2,1};
2 auto [ls,rs]=cartesian(a);
3 int rt=min_element(a.begin(),a.end())-a.begin();
4 assert(rt==1 && ls[rt]==0 && rs[rt]==3 && ls[3]==2);
```

使用边界

空输入返回两个空 vector，此时没有根。构造无递归，单调数组形成长链也可直接构建；后续遍历要自行避免递归爆栈。类型只需支持小于比较。返回值不含原数组或额外哨兵。

完整代码

```
1 template<typename T> pair<vector<int>,vector<int>> cartesian(const vector<T>
2     &a){
3     assert(a.size()<(size_t)INT_MAX);int n=a.size();
4     vector<int> ls(n,-1),rs(n,-1),st;st.reserve(n);
5     for(int i=0;i<n;i++){
6         int last=-1;
7         while(!st.empty()&&a[i]<a[st.back()]){last=st.back();st.pop_back();}
8         ls[i]=last;
9         if(!st.empty())rs[st.back()]=i;
10        st.push_back(i);
11    }
12    return {move(ls),move(rs)};
13 }
```

5 数论

5.1 number theory/basic.cpp

47 模逆元和扩展 CRT

什么时候用

提供 gcd、lcm、逆元与不要求模数互素的同余方程合并。

调用顺序与接口

- `inv(a,m)` 要求 $m>0$ 且 $\gcd(a,m)=1$ 。
- `COE(r,m)` 表示 $x=r \pmod m$ ， $a+b$ 或 $a+=b$ 合并两个方程。
- 结果 `empty()` 为 `true` 表示无解；否则 `r` 是标准余数、`M` 是合并模数，`accept(x)` 判断满足性。

最小示例（放入 main）：

```
1 COE a(2,3),b(3,5); auto c=a+b;
2 assert(!c.empty() && c.r==8 && c.M==15);
3 assert(c.accept(23) && inv(3,11)==4);
4 assert((COE(0,2)+COE(1,2)).empty());
```

使用边界

合并后的模数须能放入 `ll`，内部乘法虽提升到 `__int128`，但结果不是任意精度。`inv` 函数会与部分模数模板的全局 `inv` 数组重名，需要按实际组合改名。

完整代码

```
1 using ull=unsigned long long;
2 inline ull gcd(ull x,ull y){
3     // Stein's Algorithm
4     if(!x||!y) return x|y;
5     int c=__builtin_ctzll(x|y);
6     x>>=__builtin_ctzll(x),y>>=__builtin_ctzll(y);
7     while(x!=y){
8         if(x<y)swap(x,y);
9         x-=y;
10        x>>=__builtin_ctzll(x);
11    }
12    return y<<c;
13 }
14 inline ull lcm(ull x,ull y){
15     if(!x||!y)return 0;
16     x/=gcd(x,y);assert(x<=ULLONG_MAX/y);return x*y;
17 }
18 inline ll inv(ll x,ll M){
19     // return x^{-1} mod M
20     assert(M>0);x%=M;if(x<0)x+=M;
21     ull a=M,b=x,__int128 y=0,z=1;
22     while(b){
23         const ull q=a/b,c=a-q*b;
24         a=b,b=c;
25         const __int128 w=y-static_cast<__int128>(q)*z;
26         y=z,z=w;
27     }
28     assert(a==1U);y%=M;
29     return (y>=0?y:M+y);
```

```

30 }
31 // Congruence equation, make, merge, etc.
32 struct COE{
33     ll r,M;
34     COE():r(0),M(1){}
35     COE(ll _r,ll _M):r(_r),M(_M){assert(M>0);r%=M;if(r<0)r+=M;}
36     inline bool empty()const{return M==1;}
37     inline bool accept(ll x)const{if(empty())return false;x%=M;return
38         (x<0?x+M:x)==r;}
39     inline COE& operator+=(const COE &x){
40         if(empty()||x.empty()){M=-1;return *this;}
41         ll g=gcd(M,x.M);
42         if(r%g!=x.r%g){M=-1;return *this;}
43         assert(M/g<=LLONG_MAX/x.M);
44         ll i=inv(M/g,x.M/g),new_M=M/g*x.M;
45         r=((__int128)r+static_cast<__int128>(x.r-r)/g*i%(x.M/g)*M)%new_M;
46         M=new_M;
47         if(r<0) r+=M;
48         return *this;
49     }
50     inline COE operator+(const COE &x)const{
51         return COE(*this)+=x;
52     }
53 };

```

5.2 number theory/pollard-rho.cpp

49 大整数判素与质因数分解

什么时候用

Miller-Rabin 配合 Pollard-Rho，适用于范围内不适合筛到上界的整数。随机算法的运行时间依赖输入。

调用顺序与接口

- `is_prime(n)` 判断素数。
- `factor(n)` 返回升序质因子，含重数；`n=1` 返回空。
- `pollard_rho(n)` 提供一个因子，素数时返回自身。

最小示例（放入 main）：

```

1 auto f=factor(360);
2 assert((f==vector<ull>{2,2,2,3,3,5}));
3 assert(is_prime(1000000007ULL));

```

使用边界

`factor` 的输入要求 $1 \leq n < 2^{62}$ ，不能传 0。不是完整 `uint64` 范围版本。判素函数与线性筛的 `is_prime` 数组同名，不能原样同贴。

完整代码

```

1 using ull=unsigned long long;
2 using u128=__uint128_t;
3 inline ull rho_gcd(ull x,ull y){
4     // Stein's Algorithm
5     if(!x||!y) return x|y;

```

```

6   int c=__builtin_ctzll(x|y);
7   x>>=__builtin_ctzll(x),y>>=__builtin_ctzll(y);
8   while(x!=y){
9       if(x<y)swap(x,y);
10      x-=y;
11      x>>=__builtin_ctzll(x);
12  }
13  return y<<c;
14 }
15 bool is_prime(ull n){
16     if(n<2)return false;
17     for(auto y:{2,3,5}){
18         if(n==y)return true;
19         if(n%y==0)return false;
20     }
21     assert(n<(1ull<<62));// use Montgomery
22     ull n2=n*2,r=n&3;
23     for(int _=0;_<5;_++)r*=2-n*r;
24     r=-r;
25     ull t=-n%n,e=-u128(n)%n;
26     auto redc=[&](u128 x)->ull{return (x+u128((ull)(x)*r)*n)>>64;};
27     auto mul=[&](ull x,ull y)->ull{return redc(u128(x)*y);};
28     auto de=[&](ull x)->ull{x=redc(x);return x<n?x:x-n;};
29     auto en=[&](ull x)->ull{return mul(x,e);};
30     auto pow=[&](ull a,ull b){
31         ull res=t,base=en(a);
32         while(b){
33             if(b&1)res=mul(res,base);
34             base=mul(base,base);
35             b>>=1;
36         }
37         return res;
38     };
39     ull d=n-1;int z=__builtin_ctzll(d);d>>=z;
40     auto miller_rabin=[&](ull b){
41         if(b==0)return true;
42         ull y=pow(b,d);
43         if(de(y)==1)return true;
44         for(int i=0;i<z;i++){
45             if(de(y)==n-1)return true;
46             y=mul(y,y);
47         }
48         return false;
49     };
50     if(n<4759123141ull){
51         for(auto b:{2,7,61})if(!miller_rabin(b%n))return false;
52     }else{
53         for(auto b:{2,325,9375,28178,450775,9780504,1795265022})
54             if(!miller_rabin(b%n))return false;
55     }
56     return true;
57 }
58 ull pollard_rho(ull p){
59     assert(p>=2);
60     if(p%2==0)return 2;

```

```

61  if(is_prime(p))return p;
62  assert(p<(1ull<<62)); // use Montgomery
63  ull n=p,n2=n*2,r=n&3;
64  for(int _=0;_<5;_++)r*=2-n*r;
65  r=-r;
66  ull t=-n%n;
67  auto redc=[&](u128 x)->ull{return (x+u128((ull)(x)*r)*n)>>64;};
68  auto mul=[&](ull x,ull y)->ull{return redc(u128(x)*y);};
69  auto dif=[&](ull x,ull y)->ull{x+=n2-y;return x<n2?x:x-n2;};
70  auto de=[&](ull x)->ull{x=redc(x);return x<n?x:x-n;};
71  mt19937_64 rnd;
72  while(1){
73      ull c=rnd()%(p-1)+1,y=rnd()%(p-1)+1;
74      auto f=[&](ull x)->ull{return redc(u128(x)*x+c);};
75      for(ull s=1;;s<=1){
76          ull x=y;
77          const ull m=min(1ull<<max(int(__lg(p))/3-8,0),s);
78          for(ull i=0;i<s/m;i++){
79              ull w=t,z=y;
80              for(ull j=0;j<m;j++){
81                  y=f(y);
82                  w=mul(w,dif(y,x));
83              }
84              ull g=rho_gcd(de(w),n);
85              if(g>1){
86                  if(g<n)return g;
87                  for(ull j=0;j<m;j++){
88                      z=f(z);
89                      if((g=rho_gcd(de(dif(z,x)),n))!=1){
90                          if(g<n)return g;
91                          else goto fail;
92                      }
93                  }
94              }
95          }
96      }
97      fail;;
98  }
99 }
100 vector<ull> factor(ull x){
101     vector<ull> ans;
102     auto dfs=[&](auto self,ull x){
103         if(x==1)return;
104         ull y=pollard_rho(x);
105         if(x==y){ans.push_back(x);return;}
106         self(self,y);self(self,x/y);
107     };dfs(dfs,x);
108     sort(ans.begin(),ans.end());
109     return ans;
110 }

```

5.3 number theory/floor sum.cpp

50 类欧几里得整除求和

什么时候用

计算闭区间 $i=l..r$ 上 $\text{floor}((a*i+b)/c)$ 的和, 时间 $O(\log c)$, 返回 `__int128`。

调用顺序与接口

- `floor_sum(l,r,a,b,c)` 参数都是 `ll`。
- `l,r,a,b` 均非负, `c>0`; `l>r` 返回 0。

最小示例 (放入 main):

```
1 auto ans=floor_sum(1,4,3,1,2);
2 assert(ans==16); // 2+3+5+6
```

使用边界

不能直接处理负系数或负下标。输出可配 `int128_IO`; 不要先在 `ll` 中计算 `a*l`。答案和内部累计值须在有符号 128 位范围内。

完整代码

```
1 __int128 floor_sum(ll l,ll r,ll a,ll b,ll c){
2     assert(l>=0&&r>=0&&a>=0&&b>=0&&c>0);
3     if(l>r)return 0;
4     __int128 n=((__int128)r-l+1,A=a,B=((__int128)a*l+b,C=c,ans=0;
5     while(1){
6         ans+=n*(n-1)/2*(A/C)+n*(B/C);
7         A%=C;B%=C;
8         __int128 y=A*n+B;
9         if(y<C)break;
10        n=y/C;B=y%C;swap(A,C);
11    }
12    return ans;
13 }
```

5.4 number theory/du jiao sieve.cpp

51 μ 、 φ 前缀和

什么时候用

预筛小范围并缓存整除分块递归, 分别查询 $\text{sum } \mu(i)$ 、 $\text{sum } \varphi(i)$, $i=1..n$ 。

调用顺序与接口

- `du_jiao d`; `d.init_mu(B)` 后调用 `sum_mu(n)`, 返回 `ll`。
- `init_phi(B)` 后调用 `sum_phi(n)`, 返回 `__int128`。
- `B>=1`, `n>=0`; 两个初始化和缓存相互独立, 可只使用一个。

最小示例 (放入 main):

```
1 du_jiao d; d.init_mu(3);
2 assert(d.sum_mu(10)==-1);
3 d.init_phi(3); assert(d.sum_phi(10)==32);
4 assert(d.sum_mu(0)==0 && d.sum_phi(0)==0);
```

使用边界

必须先初始化对应函数。B 是预处理上界而非查询上限，常从 $n^{2/3}$ 附近结合内存选择；预筛 $O(B)$ ，递归性能随 B 和查询集改变。重复初始化会清空该项缓存。

完整代码

```

1 struct du_jiao{
2     vector<int> mu;
3     vector<ll> phi;
4     unordered_map<ll,ll> mem_mu;
5     unordered_map<ll,__int128> mem_phi;
6     void init_mu(int m){
7         assert(m>=1&&m<INT_MAX);
8         vector<int> prime;vector<bool> vis(m+1,false);
9         mu.assign(m+1,0);mem_mu.clear();mu[1]=1;
10        for(int i=2;i<=m;i++){
11            if(!vis[i])prime.push_back(i),mu[i]=-1;
12            for(int p:prime){
13                if(p>m/i)break;
14                int x=i*p;vis[x]=true;
15                if(i%p==0)break;
16                mu[x]=-mu[i];
17            }
18        }
19        for(int i=1;i<=m;i++)mu[i]+=mu[i-1];
20    }
21    void init_phi(int m){
22        assert(m>=1&&m<INT_MAX);
23        vector<int> prime;vector<bool> vis(m+1,false);
24        phi.assign(m+1,0);mem_phi.clear();phi[1]=1;
25        for(int i=2;i<=m;i++){
26            if(!vis[i])prime.push_back(i),phi[i]=i-1;
27            for(int p:prime){
28                if(p>m/i)break;
29                int x=i*p;vis[x]=true;
30                if(i%p==0){phi[x]=phi[i]*p;break;}
31                phi[x]=phi[i]*(p-1);
32            }
33        }
34        for(int i=1;i<=m;i++)phi[i]+=phi[i-1];
35    }
36    ll sum_mu(ll n){
37        assert(n>=0&&!mu.empty());
38        if(n<(ll)mu.size())return mu[n];
39        auto it=mem_mu.find(n);
40        if(it!=mem_mu.end())return it->second;
41        __int128 ans=1;
42        for(ll l=2,r;l<=n;){
43            r=n/(n/l);ans-=(__int128)(r-l+1)*sum_mu(n/l);
44            if(r==n)break;
45            l=r+1;
46        }
47        return mem_mu[n]=(ll)ans;
48    }
49    __int128 sum_phi(ll n){
50        assert(n>=0&&!phi.empty());

```

```
51  if(n<(ll)phi.size())return phi[n];
52  auto it=mem_phi.find(n);
53  if(it!=mem_phi.end())return it->second;
54  __int128 ans=(__int128)n*(n+(__int128)1)/2;
55  for(ll l=2,r;l<=n;){
56      r=n/(n/l);ans-=(__int128)(r-l+1)*sum_phi(n/l);
57      if(r==n)break;
58      l=r+1;
59  }
60  return mem_phi[n]=ans;
61 }
62 };
```

6 多项式

6.1 polynomial/poly-fast.cpp

52 998244353 下的常用多项式

什么时候用

适用于固定模数下的大规模卷积与形式幂级数。卷积 $O(n \log n)$ ，BM 和普通插值为二次复杂度。

调用顺序与接口

- `poly f={a0,a1,...}` 表示 $a_0+a_1x+\dots$ ，下标就是次数。
- `f*g` 是卷积；加减和数乘也支持。`Inv(f)` 返回模 x^n 的倒数， $n=f.size()$ ，`f[0]` 必须非零。
- `Ln(f)` 要求 `f[0]=1`；`Exp(f)` 要求 `f[0]=0`；返回长度与输入相同。
- `diff/integ` 求导/积分，积分常数取零；`value(f,x)` 是单点求值。
- `BM(seq)` 求线性递推的分母多项式，`FSPE(F,G,k)` 求 F/G 的第 k 项，`RSPE(seq,k)` 从已有递推样本推第 k 项， k 从 0 开始。

抄写顺序：先抄 09 的固定模数及组合数，取 $MOD=998244353$ ，再抄本节全部代码。

最小示例（放入 main）：

```
1 poly f{1,1,0,0};
2 auto g=Inv(f); auto h=f*g; h.resize(4);
3 assert(h[0].x==1 && !h[1] && !h[2] && !h[3]);
4 auto e=Exp(Ln(f));
5 for(int i=0;i<4;i++)assert(e[i].x==f[i].x);
6 assert(FSPE({0,1},{1,-1,-1},10).x==55);
```

使用边界

精度由 vector 长度决定。卷积结果长度最多 2^{23} ，FPS 输入最多 2^{22} ；ln/exp 还要求长度小于质数模数。operator/ 是相关运算，不是多项式长除。BM 返回 $C[0]=1$ 的递推分母，阶数为 $C.size()-1$ ，末尾零系数不能删去；仅拟合已给样本，外推需要足够样本。lagrange 要求互异取值点，允许零点。

完整代码

```
1 using poly=vector<mint>;
2 using Poly=poly;
3 const int FFT_MAX=23;
4 // Shared NTT; use in both the 998244353 and CRT versions.
5 template<unsigned P>
6 void ntt(vector<ModInt<P>> &a,bool rev=false){
7     using num=ModInt<P>;
8     int n=a.size();
9     assert(n&&! (n&(n-1))&&(P-1)%n==0&&n<=(1<<FFT_MAX));
10    for(int i=1,j=0;i<n;i++){
11        int k=n>>1;
12        for(;j&k;k>>=1)j^=k;
13        j^=k;if(i<j)swap(a[i],a[j]);
14    }
15    static vector<num> w{0,1};
16    for(int m=w.size();m<n;m<=<=1){
17        w.resize(2*m);num z=num(3).pow((P-1)/(2*m));
18        for(int i=m/2;i<m;i++)w[2*i]=w[i],w[2*i+1]=w[i]*z;
19    }
20    for(int m=1;m<n;m<=<=1)
```

```

21     for(int i=0;i<n;i+=2*m)
22         for(int j=0;j<m;j++){
23             num x=a[i+j],y=a[i+j+m]*w[m+j];
24             a[i+j]=x+y;a[i+j+m]=x-y;
25         }
26     if(rev){
27         reverse(a.begin()+1,a.end());num v=num(n).inv();
28         for(num &x:a)x*=v;
29     }
30 }
31 // 998244353 convolution. Replace this function by the MTT core if needed.
32 poly operator*(poly f,poly g){
33     if(f.empty()||g.empty())return {};
34     int m=f.size()+g.size()-1,n=1;
35     assert(m<=(1<<FFT_MAX));
36     if(min(f.size(),g.size())<=32){
37         poly h(m);
38         for(int i=0;i<(int)f.size();i++)
39             for(int j=0;j<(int)g.size();j++)h[i+j]+=f[i]*g[j];
40         return h;
41     }
42     while(n<m)n<=1;
43     f.resize(n);g.resize(n);ntt(f);ntt(g);
44     for(int i=0;i<n;i++)f[i]*=g[i];
45     ntt(f,true);f.resize(m);return f;
46 }
47
48 // Shared FPS; also used with the MTT convolution.
49 // Transposed multiplication: (f/g)[i] = sum_j f[i+j]*g[j].
50 poly operator/(const poly &f,poly g){
51     if(f.empty()||g.empty())return poly(f.size());
52     int m=g.size();reverse(g.begin(),g.end());g=f*g;
53     return poly(g.begin()+m-1,g.end());
54 }
55 void operator+=(poly &f,const poly &g){
56     if(f.size()<g.size())f.resize(g.size());
57     for(int i=0;i<(int)g.size();i++)f[i]+=g[i];
58 }
59 poly operator+(poly f,const poly &g){f+=g;return f;}
60 void operator-=(poly &f,const poly &g){
61     if(f.size()<g.size())f.resize(g.size());
62     for(int i=0;i<(int)g.size();i++)f[i]-=g[i];
63 }
64 poly operator-(poly f,const poly &g){f-=g;return f;}
65 poly operator*(poly f,mint x){for(auto &y:f)y*=x;return f;}
66 poly operator*(mint x,poly f){for(auto &y:f)y*=x;return f;}
67 mint value(const poly &f,mint x){
68     mint ans=0;for(int i=(int)f.size()-1;i>=0;i--)ans=ans*x+f[i];return ans;
69 }
70 poly Inv(const poly &f){
71     assert(!f.empty()&&f[0]&&f.size()<=(1<<(FFT_MAX-1)));
72     int n=f.size();poly g{f[0].inv()};
73     while((int)g.size()<n){
74         int m=min(n,(int)g.size()*2);
75         poly h=poly(f.begin(),f.begin()+m)*g;h.resize(m);

```

```

76     for(auto &x:h)x=-x;
77     h[0]+=2;g=g*h;g.resize(m);
78 }
79 return g;
80 }
81 poly diff(poly f){
82     if(f.empty())return {};
83     for(int i=1;i<(int)f.size();i++)f[i-1]=f[i]*i;
84     f.pop_back();return f;
85 }
86 poly integ(poly f){
87     int n=f.size();init(n);f.resize(n+1);
88     for(int i=n;i;i--)f[i]=f[i-1]*inv[i];
89     f[0]=0;return f;
90 }
91 poly Ln(const poly &f){
92     assert(!f.empty()&&f[0].x==1&&f.size()<(size_t)MOD&&f.size()<=(1<<(FFT_MAX-1)));
93     poly g=diff(f)*Inv(f);g.resize(f.size()-1);return integ(move(g));
94 }
95 poly Exp(const poly &f){
96     assert(!f.empty()&&!f[0]&&f.size()<(size_t)MOD&&f.size()<=(1<<(FFT_MAX-1)));
97     int n=f.size();poly g{1};
98     while((int)g.size()<n){
99         int m=min(n,(int)g.size()*2);poly h=g;h.resize(m);h=Ln(h);
100         for(int i=0;i<m;i++)h[i]=f[i]-h[i];
101         h[0]+=1;g=g*h;g.resize(m);
102     }
103     return g;
104 }
105 poly BM(const poly &a){
106     poly c{1},b{1};
107     int len=0,shift=1;
108     mint inv_d=1;
109     for(int i=0;i<(int)a.size();i++){
110         mint d=a[i];
111         for(int j=1;j<=len;j++)d+=c[j]*a[i-j];
112         if(!d){shift++;continue;}
113         bool grow=2*len<=i;
114         poly old;
115         if(grow)old=c;
116         mint v=d*inv_d;
117         if(c.size()<b.size()+shift)c.resize(b.size()+shift);
118         for(int j=0;j<(int)b.size();j++)c[j+shift]-=v*b[j];
119         if(grow){
120             len=i+1-len;b=move(old);inv_d=d.inv();shift=1;
121         }else shift++;
122     }
123     c.resize(len+1);
124     return c;
125 }
126
127 // Return  $[x^k] f/g$ ;  $g[0]$  must be invertible.
128 mint FSPE(poly f,poly g,ll k){
129     assert(k>=0&&!g.empty()&&g[0]);
130     while(k&&!f.empty()){

```

```

131     poly h=g;
132     for(int i=1;i<(int)h.size();i+=2)h[i]=-h[i];
133     f=f*h;g=g*h;
134     int n=(f.size()+1-(k&1))/2,m=(g.size()+1)/2;
135     for(int i=0;i<n;i++)f[i]=f[2*i+(k&1)];
136     for(int i=0;i<m;i++)g[i]=g[2*i];
137     f.resize(n);g.resize(m);k>>=1;
138 }
139 return f.empty()?mint(0):f[0]/g[0];
140 }
141
142 mint RSPE(poly f,ll x){
143     // Infer a recurrence, then return f[x].
144     assert(x>=0);
145     if(x<(ll)f.size())return f[x];
146     poly g=BM(f);
147     f.resize(g.size()-1);
148     poly s=f*g;s.resize(g.size()-1);
149     return FSPE(move(s),move(g),x);
150 }
151
152 poly lagrange(const vector<mint> &x,const vector<mint> &y){
153     int n=x.size();assert(y.size()==x.size());
154     poly g(n+1),f(n),q(n);g[0]=1;
155     for(int i=0;i<n;i++){
156         for(int j=i+1;j;j--)g[j]=g[j-1]-x[i]*g[j];
157         g[0]*=-x[i];
158     }
159     for(int i=0;i<n;i++){
160         q[n-1]=g[n];
161         for(int j=n-2;j>=0;j--)q[j]=g[j+1]+x[i]*q[j+1];
162         mint d=value(q,x[i]);assert(d);
163         mint c=y[i]/d;
164         for(int j=0;j<n;j++)f[j]+=q[j]*c;
165     }
166     return f;
167 }

```

6.2 polynomial/MTT.cpp

55 非 NTT 模数下的多项式

什么时候用

默认目标模数 10^9+7 ，三模 NTT 后 CRT 还原；常用 FPS 功能与 poly-fast 对齐，但每次卷积都会回到目标模数。

调用顺序与接口

- $\text{poly } f=\{a_0,a_1,\dots\}$ 表示 $a_0+a_1x+\dots$ ，下标就是次数。
- $f*g$ 是卷积；加减和数乘也支持。 $\text{Inv}(f)$ 返回模 x^n 的倒数， $n=f.size()$ ， $f[0]$ 必须非零。
- $\text{Ln}(f)$ 要求 $f[0]=1$ ； $\text{Exp}(f)$ 要求 $f[0]=0$ ；返回长度与输入相同。
- diff/integ 求导/积分，积分常数取零； $\text{value}(f,x)$ 是单点求值。
- $\text{BM}(\text{seq})$ 求线性递推的分母多项式， $\text{FSPE}(F,G,k)$ 求 F/G 的第 k 项， $\text{RSPE}(\text{seq},k)$ 从已有递推样本推第 k 项， k 从 0 开始。

- Poly 与 poly 是同一个类型；lagrange(xs,ys) 支持互异取值点，包括零点，返回次数小于点数的插值多项式。

抄写顺序：先抄 09，把 MOD 改为目标模数（如 1000000007）；抄 52 开头的 poly/Poly、FFT_MAX 与 ntt；随后抄本节卷积；最后抄 52 标注 Shared FPS 的部分。不要同时抄 52 的 operator* 卷积。

最小示例（放入 main）：

```
1 poly f{1,1,0,0};
2 auto g=Inv(f); auto h=f*g; h.resize(4);
3 assert(h[0].x==1 && !h[1] && !h[2] && !h[3]);
4 auto e=Exp(Ln(f));
5 for(int i=0;i<4;i++)assert(e[i].x==f[i].x);
6 assert(FSPE({0,1},{1,-1,-1},10).x==55);
7 auto p=lagrange({0,1,2},{1,4,9});
8 assert(p[0].x==1 && p[1].x==2 && p[2].x==1);
```

使用边界

精度由 vector 长度决定。卷积结果长度最多 2^{23} ，FPS 输入最多 2^{22} ；ln/exp 还要求长度小于质数模数。operator/ 是相关运算，不是多项式长除。BM 返回 $C[0]=1$ 的递推分母，阶数为 $C.size()-1$ ，末尾零系数不能删去；仅拟合已给样本，外推需要足够样本。lagrange 要求互异取值点，允许零点。

完整代码

```
1 template<unsigned P>
2 vector<ModInt<P>> convolution(const poly &f,const poly &g,int n){
3     vector<ModInt<P>> a(n),b(n);
4     for(int i=0;i<(int)f.size();i++)a[i]=f[i].x;
5     for(int i=0;i<(int)g.size();i++)b[i]=g[i].x;
6     ntt(a);ntt(b);
7     for(int i=0;i<n;i++)a[i]*=b[i];
8     ntt(a,true);a.resize(f.size()+g.size()-1);return a;
9 }
10 poly operator*(const poly &f,const poly &g){
11     if(f.empty()||g.empty())return {};
12     int m=f.size()+g.size()-1,n=1;
13     assert(m<=(1<<FFT_MAX));
14     poly h(m);
15     if(min(f.size(),g.size())<=32){
16         for(int i=0;i<(int)f.size();i++)
17             for(int j=0;j<(int)g.size();j++)h[i+j]+=f[i]*g[j];
18         return h;
19     }
20     constexpr unsigned p1=998244353,p2=167772161,p3=469762049;
21     constexpr ull p12=1ull*p1*p2;
22     static const unsigned v1=ModInt<p2>(p1).inv().x;
23     static const unsigned v2=ModInt<p3>(p12).inv().x;
24     while(n<m)n<=1;
25     auto a=convolution<p1>(f,g,n);
26     auto b=convolution<p2>(f,g,n);
27     auto c=convolution<p3>(f,g,n);
28     for(int i=0;i<m;i++){
29         ull x=(b[i].x+p2-a[i].x%p2)*1ull*v1%p2;
30         x=x*p1+a[i].x;
31         ull y=(c[i].x+p3-x%p3)*v2%p3;
32         h[i]=x%MOD+(p12%MOD)*y%MOD;
33     }
34     return h;
```

35 }

6.3 polynomial/FWT.cpp

56 按位 AND / OR / XOR 卷积

什么时候用

求 $c_k = \sum_{i \oplus j = k} a_i b_j$ ，其中 op 为按位 AND、OR 或 XOR。长度 $n = 2^k$ ，每次变换 $O(n \log n)$ ，额外空间 $O(1)$ 。

调用顺序与接口

- `convolution_and(a,b)`、`convolution_or(a,b)`、`convolution_xor(a,b)` 直接返回对应卷积，不修改传入数组。支持长度不同、非二次幂长度；任一输入为空则返回空数组，否则结果长度为不小于两者最大长度的最小二次幂。
- `fwt_and(a)`、`fwt_or(a)`、`fwt_xor(a)` 原地正变换；第二个参数传 `true` 做逆变换。
- 支持 `vector<int>`、`vector<ll>` 和 `vector<mint>`。mint 需先抄模整数定义，FWT 本身不依赖多项式文件。
- 两个数组补零到相同的非零二次幂长度 n ，使 n 大于最大下标；不必补到两者长度之和。
- 两边做同一种正变换，逐点相乘，再做对应逆变换，即得卷积。

最小示例（放入 main）：

```
1 vector<int> a{1,2},b{3,4};
2 auto c=convolution_and(a,b); assert((c==vector<int>{13,8}));
3 c=convolution_or(a,b); assert((c==vector<int>{3,18}));
4 c=convolution_xor(a,b); assert((c==vector<int>{11,10}));
5 // vector<mint> works with the same interfaces.
```

使用边界

下标从 0 开始。int/ll 表示普通整数运算，不会自动取模，正变换和逐点乘法的中间值也必须不溢出。整数 XOR 逆变换逐层除以 2，要求每步可整除，不能把任意整数数组当成合法频域数据。XOR 模运算要求 2 可逆，通常使用奇数模数；AND/OR 无此限制。不需要归一化第二次。

完整代码

```
1 template<typename T>
2 void fwt_and(vector<T> &a,bool inverse=false){
3     assert(!a.empty()&&a.size()<=INT_MAX&&!(a.size()&(a.size()-1)));
4     int n=a.size();
5     for(int k=1;k<n;k<=<=1)
6         for(int i=0;i<n;i+=k<<1)
7             for(int j=0;j<k;j++){
8                 if(inverse)a[i+j]-=a[i+j+k];
9                 else a[i+j]+=a[i+j+k];
10            }
11 }
12 template<typename T>
13 void fwt_or(vector<T> &a,bool inverse=false){
14     assert(!a.empty()&&a.size()<=INT_MAX&&!(a.size()&(a.size()-1)));
15     int n=a.size();
16     for(int k=1;k<n;k<=<=1)
17         for(int i=0;i<n;i+=k<<1)
18             for(int j=0;j<k;j++){
19                 if(inverse)a[i+j+k]-=a[i+j];
20                 else a[i+j+k]+=a[i+j];
21            }
22 }
```

```

22 }
23 template<typename T>
24 void fwt_xor(vector<T> &a,bool inverse=false){
25     assert(!a.empty()&&a.size()<=INT_MAX&&!(a.size()&(a.size()-1)));
26     int n=a.size();T inv2=1;
27     if constexpr(!is_integral_v<T>)if(inverse)inv2=T(1)/T(2);
28     for(int k=1;k<n;k<=<1)
29         for(int i=0;i<n;i+=k<1)
30             for(int j=0;j<k;j++){
31                 if constexpr(is_integral_v<T>){
32                     using W=conditional_t<(sizeof(T)<=4),long long,__int128>;
33                     W x=a[i+j],y=a[i+j+k],u=x+y,v=x-y;
34                     if(inverse){assert(u%2==0&&v%2==0);u/=2;v/=2;}
35                     assert(numeric_limits<T>::lowest()<=u&&u<=numeric_limits<T>::max());
36                     assert(numeric_limits<T>::lowest()<=v&&v<=numeric_limits<T>::max());
37                     a[i+j]=T(u);a[i+j+k]=T(v);
38                 }else{
39                     T x=a[i+j],y=a[i+j+k];
40                     a[i+j]=x+y;a[i+j+k]=x-y;
41                     if(inverse){a[i+j]*=inv2;a[i+j+k]*=inv2;}
42                 }
43             }
44 }
45
46 template<typename T>
47 vector<T> convolution_and(vector<T> a,vector<T> b){
48     if(a.empty()||b.empty())return {};
49     assert(max(a.size(),b.size())<=(1u<<30));
50     int n=1;while(n<(int)max(a.size(),b.size()))n<=<1;
51     a.resize(n);b.resize(n);
52     fwt_and(a);fwt_and(b);
53     for(int i=0;i<n;i++)a[i]*=b[i];
54     fwt_and(a,true);return a;
55 }
56
57 template<typename T>
58 vector<T> convolution_or(vector<T> a,vector<T> b){
59     if(a.empty()||b.empty())return {};
60     assert(max(a.size(),b.size())<=(1u<<30));
61     int n=1;while(n<(int)max(a.size(),b.size()))n<=<1;
62     a.resize(n);b.resize(n);
63     fwt_or(a);fwt_or(b);
64     for(int i=0;i<n;i++)a[i]*=b[i];
65     fwt_or(a,true);return a;
66 }
67
68 template<typename T>
69 vector<T> convolution_xor(vector<T> a,vector<T> b){
70     if(a.empty()||b.empty())return {};
71     assert(max(a.size(),b.size())<=(1u<<30));
72     int n=1;while(n<(int)max(a.size(),b.size()))n<=<1;
73     a.resize(n);b.resize(n);
74     fwt_xor(a);fwt_xor(b);
75     for(int i=0;i<n;i++)a[i]*=b[i];
76     fwt_xor(a,true);return a;

```

77 }

7 字符串

7.1 string/sa.cpp

57 后缀数组和 $O(1)$ LCP

什么时候用

计数排序倍增求后缀数组;SA_LCP 由 height 与四毛子 RMQ 查询两后缀的最长公共前缀。先抄 03 的 Linear_RMQ。

调用顺序与接口

- SA sa; sa.build(" "+s) 输入前面补一个占位空格，真实字符下标 1..n。
- sa.sa[r] 是排名第 r 的后缀起点; sa.rk[i] 是后缀 i 的排名。
- SA_LCP l; l.build(sa), 然后 l.lcp(i,j) 或 l.ask(i,j) 查询, i,j 是原串位置。
- 也可 l.build(" "+s) 直接建。height[r] 为排名 r 与 r-1 的后缀 LCP, height[1]=0。

最小示例（放入 main）:

```
1 SA sa; sa.build(" banana");
2 assert(sa.sa[1]==6);
3 SA_LCP l; l.build(sa);
4 assert(l.lcp(2,4)==3); // anana / ana
5 assert(l.lcp(3,3)==4);
```

使用边界

SA 构建 $O(n \log n)$, 工作空间 $O(n)$, LCP 结构构建与空间 $O(n)$, 查询 $O(1)$ 。不要把后缀排名传给 lcp。默认字符编码范围 1..127, 任意非零字节传 $M=256$; 不支持串内零字节。修改原串后须重建两者。

完整代码

```
1 struct SA{
2     int n=0,m=0;
3     vector<int> sa,rk;
4     vector<unsigned char> s;
5     void set(){n=m=0;sa.clear();rk.clear();s.clear();}
6     void build(string S,int M=128){
7         assert(!S.empty()&&S[0]!=' ' &&1<=M&&M<=256);
8         assert(S.size()<INT_MAX/2);
9         n=S.size()-1;m=M;
10        sa.assign(n+1,0);rk.assign(n+1,0);s.assign(n+1,0);
11        vector<int> y(n+1),old(n+1),cnt(max(n+1,M),0);
12        for(int i=1;i<=n;i++){
13            s[i-1]=(unsigned char)S[i];
14            assert(0<s[i-1]&&s[i-1]<M);
15            rk[i]=s[i-1];cnt[rk[i]]++;
16        }
17        for(int i=1;i<=m;i++)cnt[i]+=cnt[i-1];
18        for(int i=n;i;i--)sa[cnt[rk[i]]--]=i;
19        for(int k=1;k<=n;k<=1){
20            int p=0;
21            for(int i=n-k+1;i<=n;i++)y[++p]=i;
22            for(int i=1;i<=n;i++)if(sa[i]>k)y[++p]=sa[i]-k;
23            fill(cnt.begin(),cnt.end(),0);
24            for(int i=1;i<=n;i++)cnt[rk[i]]++;
25            for(int i=1;i<=m;i++)cnt[i]+=cnt[i-1];
26            for(int i=n;i;i--)sa[cnt[rk[y[i]]]--]=y[i];
```

```

27     old=rk;p=0;
28     for(int i=1;i<=n;i++){
29         int x=sa[i],z=sa[i-1];
30         if(i==1||old[x]!=old[z]||
31             (x+k<=n?old[x+k]:0)!=(z+k<=n?old[z+k]:0))p++;
32         rk[x]=p;
33     }
34     m=p+1;if(p==n)break;
35 }
36 for(int i=1;i<=n;i++)rk[sa[i]]=i;
37 }
38 void output(){
39     cerr<<"string: ";for(int i=0;i<n;i++)cerr<<s[i];cerr<<"\n";
40     cerr<<"sa: ";for(int i=1;i<=n;i++)cerr<<sa[i]<<" \n"[i==n];
41     cerr<<"rk: ";for(int i=1;i<=n;i++)cerr<<rk[i]<<" \n"[i==n];
42 }
43 void solve(){for(int i=1;i<=n;i++)cout<<sa[i]<<" \n"[i==n];}
44 };
45 // SA_LCP L; L.build(sa); L.lcp(i,j); // 1-indexed
46 // O(n) LCP build, O(1) query; paste Linear_RMQ first.
47 struct SA_LCP{
48     int n=0;
49     vector<int> rk,height;
50     Linear_RMQ<int> st;
51     void build(const SA &S){
52         n=S.n;rk=S.rk;height.assign(n+1,0);st.set();
53         assert((int)S.sa.size()==n+1&&(int)rk.size()==n+1);
54         for(int i=1,k=0;i<=n;i++){
55             int r=rk[i];
56             if(r==1){k=0;continue;}
57             int j=S.sa[r-1];
58             while(i+k<=n&&j+k<=n&&S.s[i+k-1]==S.s[j+k-1])k++;
59             height[r]=k;if(k)k--;
60         }
61         st.build(height);
62     }
63     void build(const string &s,int M=128){SA S;S.build(s,M);build(S);}
64     int lcp(int i,int j)const{
65         assert(1<=min(i,j)&&max(i,j)<=n);
66         if(i==j)return n-i+1;
67         int l=rk[i],r=rk[j];if(l>r)swap(l,r);l++;
68         return st.ask(l,r);
69     }
70     int ask(int i,int j)const{return lcp(i,j);}
71 };

```

7.2 string/sam.cpp

58 单串后缀自动机

什么时候用

构建单个小写字母串的子串状态图，并统计每个状态对应子串的出现次数。时间及空间 $O(n)$ ，字母表固定 26。

调用顺序与接口

- SAM t; t.build(" "+s) 每次重新构建。
- 根为 0; ch[u][c] 为字符 c 的转移，0 表示不存在。
- 状态 u 对应长度区间 (len[fail[u]],len[u])，同一状态内的子串出现次数均为 siz[u]。

最小示例（放入 main）：

```
1 SAM t; t.build(" ababa");
2 int u=0; for(char c:string("aba"))u=t.ch[u][c-'a'];
3 assert(u && t.siz[u]==2);
4 ll distinct=0;
5 for(int i=1;i<=t.tot;i++)distinct+=t.len[i]-t.len[t.fail[i]];
6 assert(distinct==9);
```

使用边界

匹配模式串时遇到不存在转移应立即判定不出现。build 内已累计 siz，不能再次累计。裸调用 extend 不会自动维护整套出现次数；多串应用请用 GSAM。solve 是“重复子串长度乘次数最大值”的特定示例，不是通用查询。

完整代码

```
1 struct SAM{
2 // this SAM rooted at 0
3 string s;
4 int n=0,tot=0,lst=0;
5 vector<array<int,26>> ch;
6 vector<int> fail,len,siz;
7 vector<vector<int>> son;
8 SAM(){set();}
9 void set(int m=0){
10 assert(0<=m&&m<(INT_MAX-1)/2);n=tot=lst=0;
11 ch.assign(1,{});fail.assign(1,-1);len.assign(1,0);siz.assign(1,0);
12 son.assign(1,{});
13 ch.reserve(2*m+1);fail.reserve(2*m+1);len.reserve(2*m+1);
14 siz.reserve(2*m+1);son.reserve(2*m+1);
15 }
16 int new_node(){
17 assert(tot<INT_MAX-1);ch.push_back({});fail.push_back(0);
18 len.push_back(0);siz.push_back(0);son.emplace_back();return ++tot;
19 }
20 inline int clone(int x){
21 new_node();ch[tot]=ch[x];
22 fail[tot]=fail[x];
23 return tot;
24 }
25 void extend(char c){
26 assert('a'<=c&&c<='z');c-='a'; // attention the alphabet
27 int cur=new_node();len[cur]=len[lst]+1;
28 while(~lst&&!ch[lst][c]) ch[lst][c]=tot,lst=fail[lst];
29 if(lst==-1) fail[cur]=0;
```

```

30     else{
31         int p=ch[lst][c];
32         if(len[p]==len[lst]+1) fail[cur]=p;
33         else{
34             int q=clone(p);len[q]=len[lst]+1;
35             while(~lst&&ch[lst][c]==p) ch[lst][c]=q,lst=fail[lst];
36             fail[p]=fail[cur]=q;
37         }
38     }
39     lst=cur;
40 }
41 void output(){
42     // this allow you to see the structure of SAM
43     // output fail, ed, len etc.
44     cerr<<"string:"<<s<<endl;
45     cerr<<"# node = "<<tot<<endl;
46     cerr<<"fail: ";for(int i=0;i<=tot;i++) cerr<<fail[i]<<" \n"[i==tot];
47     cerr<<"len: ";for(int i=0;i<=tot;i++) cerr<<len[i]<<" \n"[i==tot];
48     // other else?
49     cerr<<"siz: ";for(int i=0;i<=tot;i++) cerr<<siz[i]<<" \n"[i==tot];
50 }
51 void dfs(int u){
52     vector<int> q{u};
53     for(int i=0;i<(int)q.size();i++)for(int v:son[q[i]])q.push_back(v);
54     for(int i=(int)q.size()-1;i>0;i--)siz[fail[q[i]]]+=siz[q[i]];
55 }
56 void build(string S){
57     assert(!S.empty()&&S[0]==' ' &&S.size()<(INT_MAX-1)/2);
58     set<(int)>S.size()-1;s=move(S);n=(int)s.size()-1;
59     for(int i=1;i<=n;i++){
60         extend(s[i]);
61         // something record here
62         siz[lst]=1;
63     }
64     for(int i=1;i<=tot;i++) son[fail[i]].push_back(i);
65     dfs(0);
66     // output();
67 }
68 void solve(){
69     // this is the main work
70     ll ans=0;
71     for(int i=1;i<=tot;i++) if(siz[i]>1) ans=max(ans,1ll*siz[i]*len[i]);
72     cout<<ans<<"\n";
73 }
74 };

```

7.3 string/ac.cpp

59 多模式串 AC 自动机

什么时候用

一次加入多个模式串，再统计它们在文本中的出现次数。允许重叠出现；构建 $O(26 \times \text{节点数})$ ，每次 count 为 $O(\text{文本长度} + \text{节点数})$ 。

调用顺序与接口

- AC ac; id=ac.insert(" "+pattern) 返回该模式串末节点编号。
- 插入完所有模式串后调用 build，之后不能再插入，除非 set 清空重建。
- auto cnt=ac.count(" "+text)，模式串答案是 cnt[id]。
- build 后 ch 是补全后的转移，可直接用于自动机 DP；fail 是失配链接。

最小示例（放入 main）：

```
1 AC ac; int a=ac.insert(" aba"),b=ac.insert(" ba");
2 ac.build(); auto cnt=ac.count(" ababa");
3 assert(cnt[a]==2 && cnt[b]==2);
```

使用边界

模式和文本都只支持小写字母，前导空格不参加匹配。重复模式串返回同一编号，但它们各自的答案仍是该模式在文本中的出现次数。空模式返回根编号 0，次数为文本长度 +1。不同 count 之间不会累计。

完整代码

```
1 struct AC{
2     int tot=0;
3     bool built=false;
4     vector<array<int,26>> ch;
5     vector<int> fail,q;
6     AC(){set();}
7     void set(int m=0){
8         assert(0<=m&&m<INT_MAX-1);tot=0;built=false;
9         ch.assign(1,{});fail.assign(1,0);q.clear();
10        ch.reserve(m+1);fail.reserve(m+1);
11    }
12    int new_node(){
13        assert(tot<INT_MAX-1);ch.push_back({});fail.push_back(0);return ++tot;
14    }
15    int insert(const string &s){
16        assert(!built&&!s.empty()&&s[0]!=' ' && s.size()<INT_MAX);
17        int u=0;
18        for(int i=1;i<(int)s.size();i++){
19            assert('a'<=s[i]&&s[i]<='z');int c=s[i]-'a';
20            if(!ch[u][c]){int v=new_node();ch[u][c]=v;}
21            u=ch[u][c];
22        }
23        return u;
24    }
25    void build(){
26        if(built)return;
27        q.clear();q.reserve(tot+1);q.push_back(0);
28        for(int i=0;i<(int)q.size();i++){
29            int u=q[i];
30            for(int c=0;c<26;c++){
```

```

31     int v=ch[u][c];
32     if(v){fail[v]=u?ch[fail[u]][c]:0;q.push_back(v);}
33     else if(u)ch[u][c]=ch[fail[u]][c];
34 }
35 }
36 built=true;
37 }
38 vector<ll> count(const string &s)const{
39     assert(built&&!s.empty()&&s[0]!=' '&&s.size()<INT_MAX);
40     vector<ll> cnt(tot+1);cnt[0]=1;int u=0;
41     for(int i=1;i<(int)s.size();i++){
42         assert('a'<=s[i]&&s[i]<='z');u=ch[u][s[i]-'a'];cnt[u]++;
43     }
44     for(int i=(int)q.size()-1;i;i--)cnt[fail[q[i]]]+=cnt[q[i]];
45     return cnt;
46 }
47 };

```

7.4 string/gsam.cpp

60 广义后缀自动机

什么时候用

维护多个小写串的所有子串；出现次数是这些串中次数之和，重复插入的串也重复计数，不允许跨串边界匹配。

调用顺序与接口

- GSAM t; t.insert(" "+s) 加入一条字符串，之后 t.build() 累计出现次数。
- count(" "+p) 返回模式的总出现次数；find(" "+p) 返回状态编号，不存在为 -1。
- distinct() 返回所有插入串合起来的不同非空子串数。
- 可继续 insert，但下一次 count 前必须重新 build。

最小示例（放入 main）：

```

1 GSAM t; t.insert(" aba"); t.insert(" bab");
2 t.build(); assert(t.count(" ab")==2);
3 assert(t.count(" abab")==0 && t.distinct()==6);
4 t.insert(" aba"); t.build(); assert(t.count(" aba")==2);

```

使用边界

count 不是“包含该子串的字符串个数”。根为 0，fail[0]=-1；别把普通 SAM 简单重置 lst 的做法当成这里的插入实现。set(m) 的 m 用于预留总字符量级的空间，build 的计数按当前全部插入内容重新计算。

完整代码

```

1 struct GSAM{
2     int n=0,tot=0;
3     ll strings=0;
4     bool built=false;
5     vector<array<int,26>> ch;
6     vector<int> fail,len;
7     vector<ll> cnt,siz;
8     GSAM(){set();}
9     void set(int m=0){
10         assert(0<=m&&m<(INT_MAX-1)/2);n=tot=0;strings=0;built=false;

```

```

11  ch.assign(1,{});fail.assign(1,-1);len.assign(1,0);cnt.assign(1,0);
12  siz.clear();
13  ch.reserve(2*m+1);fail.reserve(2*m+1);len.reserve(2*m+1);cnt.reserve(2*m+1);
14  }
15  int new_node(){
16  assert(tot<INT_MAX-1);ch.push_back({});fail.push_back(0);len.push_back(0);
17  cnt.push_back(0);return ++tot;
18  }
19  int clone(int x,int l){
20  int y=new_node();ch[y]=ch[x];fail[y]=fail[x];len[y]=l;return y;
21  }
22  int extend(int p,int c){
23  if(ch[p][c]){
24  int q=ch[p][c];if(len[q]==len[p]+1)return q;
25  int r=clone(q,len[p]+1);
26  while(p!=-1&&ch[p][c]==q)ch[p][c]=r,p=fail[p];
27  fail[q]=r;return r;
28  }
29  int cur=new_node();len[cur]=len[p]+1;
30  while(p!=-1&&!ch[p][c])ch[p][c]=cur,p=fail[p];
31  if(p==-1)fail[cur]=0;
32  else{
33  int q=ch[p][c];
34  if(len[q]==len[p]+1)fail[cur]=q;
35  else{
36  int r=clone(q,len[p]+1);
37  while(p!=-1&&ch[p][c]==q)ch[p][c]=r,p=fail[p];
38  fail[q]=fail[cur]=r;
39  }
40  }
41  return cur;
42  }
43  int insert(const string &s){
44  assert(!s.empty()&&s[0]!=' '&&s.size()<INT_MAX);
45  int u=0;n=max(n,(int)s.size()-1);strings++;built=false;
46  for(int i=1;i<(int)s.size();i++){
47  assert('a'<=s[i]&&s[i]<='z');u=extend(u,s[i]-'a');cnt[u]++;
48  }
49  return u;
50  }
51  void build(){
52  vector<int> c(n+1),q(tot+1);siz=cnt;
53  for(int i=0;i<=tot;i++)c[len[i]]++;
54  for(int i=1;i<=n;i++)c[i]+=c[i-1];
55  for(int i=0;i<=tot;i++)q[--c[len[i]]]=i;
56  for(int i=tot;i;i--)siz[fail[q[i]]]+=siz[q[i]];
57  siz[0]+=strings;built=true;
58  }
59  int find(const string &s)const{
60  assert(!s.empty()&&s[0]!=' '&&s.size()<INT_MAX);int u=0;
61  for(int i=1;i<(int)s.size();i++){
62  assert('a'<=s[i]&&s[i]<='z');u=ch[u][s[i]-'a'];if(!u)return -1;
63  }
64  return u;
65  }

```

```

66 ll count(const string &s) const {
67     assert(built); int u = find(s); return u == -1 ? 0 : siz[u];
68 }
69 ll distinct() const {
70     ll ans = 0; for (int i = 1; i <= tot; i++) ans += len[i] - len[fail[i]]; return ans;
71 }
72 };

```

7.5 string/kmp.cpp

61 前缀函数和 Z 函数

什么时候用

线性时间计算字符串的 border 与各后缀和全串的公共前缀长度。

调用顺序与接口

- `border(" "+s)` 返回 `pi`, `pi[i]` 为 `s[1..i]` 的最长真 border 长度。
- `z_function(" "+s)` 返回 `z`, `z[i]` 为 `s[i..n]` 与 `s[1..n]` 的 LCP。
- 数组长度 `n+1`, 0 号是占位。

最小示例（放入 main）:

```

1 auto pi = border(" ababa"); auto z = z_function(" ababa");
2 assert(pi[5] == 3 && z[3] == 3);

```

使用边界

这个实现 `z[1]=0`，而不是全串长度。没有封装“模式串在文本的所有位置”：可自行 KMP 扫描，或拼接 `pattern+ 分隔符 +text` 后用 `Z`；分隔符不能出现在两个原串里。

完整代码

```

1 vector<int> border(string s){
2     // s is a string with length n+1,
3     // this function return a vector with length n+1
4     // indication the border array of s
5     assert(!s.empty() && s.size() <= INT_MAX);
6     int n = (int)s.length() - 1;
7     assert(!s.empty() && s[0] == ' ');
8     vector<int> a(n+1);
9     for(int i = 2, j = 0; i <= n; i++){
10         while(j && s[j+1] != s[i]) j = a[j];
11         if(s[j+1] == s[i]) j++;
12         a[i] = j;
13     }
14     return a;
15 }
16 vector<int> z_function(string s){
17     // s is a string with length n+1,
18     // this function return a vector with length n+1
19     // indication the a[i] = lcp(s[i:], s[])
20     assert(!s.empty() && s.size() <= INT_MAX);
21     int n = (int)s.length() - 1;
22     assert(!s.empty() && s[0] == ' ');
23     vector<int> z(n+1);

```

```

24   for(int i=2,l=1,r=0;i<=n;i++){
25       z[i]=r<i?0:min(r-i+1,z[i-1+1]);
26       while(i+z[i]<=n&&s[1+z[i]]==s[i+z[i]]) z[i]++;
27       if(i+z[i]>r) l=i,r=i+z[i]-1;
28   }
29   return z;
30 }

```

7.6 string/hash.cpp

62 双模前缀哈希

什么时候用

提供子串指纹与拼接，不是无碰撞的字符串判等。构建前缀数组 $O(n)$ ，常规子串取值 $O(1)$ 。

调用顺序与接口

- `auto h=Hash_of(" "+s)` 产生前缀哈希。
- 子串 $[l,r]$ 为 `h[r]-h[l-1]`。
- `a+b` 拼接两段，`h.v()` 打包哈希值；`.l` 是长度。
- `init_hash(n)` 可预留幂次；实际操作也会自动扩容。

最小示例（放入 main）：

```

1 auto h=Hash_of(" ababa");
2 auto a=h[3]-h[0],b=h[5]-h[2];
3 assert(a.l==b.l && a.v()==b.v());

```

使用边界

比较子串时同时比较长度和哈希。底数每个进程随机，不能把数值用于跨进程永久存储。减法只表示删去前缀，不是任意删除；Hash 对象应使用 `{}` 零初始化。

完整代码

```

1 mt19937 rnd(time(0));
2 const int MOD1 = 998244353;
3 const int MOD2 = 1e9 + 7;
4 const int B1 = 256 + rnd() % (MOD1-257);
5 const int B2 = 256 + rnd() % (MOD2-257);
6
7 vector<ll> pw1{1}, pw2{1};
8 void init_hash(int n=0);
9
10 struct Hash {
11     int l=0;
12     ll h1=0,h2=0;
13     ll v() const { return h1 << 30 | h2; }
14 };
15
16 inline Hash operator+(const Hash &x, const Hash &y) {
17     // append y to end of x
18     assert(x.l>=0&&y.l>=0&&x.l<=INT_MAX-y.l);init_hash(y.l);
19     return {x.l + y.l,
20             (x.h1 * pw1[y.l] + y.h1) % MOD1,
21             (x.h2 * pw2[y.l] + y.h2) % MOD2};

```

```

22 }
23
24 inline Hash operator-(const Hash &x, const Hash &y) {
25     // y is prefix of x, delete y from x
26     assert(0<=y.l&&y.l<=x.l);init_hash(x.l-y.l);
27     return {x.l - y.l,
28         (x.h1 - y.h1 * pw1[x.l - y.l] % MOD1 + MOD1) % MOD1,
29         (x.h2 - y.h2 * pw2[x.l - y.l] % MOD2 + MOD2) % MOD2};
30 }
31
32 template<typename T>
33 inline Hash operator+(const Hash &h, const T &x) {
34     // append x to end of h
35     assert(0<=h.l&&h.l<INT_MAX);
36     return {h.l + 1,
37         (h.h1 * B1 + x) % MOD1,
38         (h.h2 * B2 + x) % MOD2};
39 }
40
41 template<typename T>
42 inline void operator+=(Hash &h, const T &x) {
43     // append x to end of h
44     h = h + x;
45 }
46
47 void init_hash(int n) {
48     assert(0<=n&&n<INT_MAX);
49     int m=pw1.size();if(n<m)return;
50     n=max(n,(int)min(2ll*m,(1ll)INT_MAX-1));
51     pw1.resize(n+1);pw2.resize(n+1);
52     for(int i=m;i<=n;i++){
53         pw1[i]=pw1[i-1]*B1%MOD1;pw2[i]=pw2[i-1]*B2%MOD2;
54     }
55 }
56
57 vector<Hash> Hash_of(const string &s) {
58     int n = (int)s.length() - 1;
59     assert(!s.empty()&&s[0]!=' '&&s.size()<=INT_MAX);init_hash(n);
60     vector<Hash> a(n+1);
61     for(int i = 1; i <= n; ++i)
62         a[i] = a[i-1] + (unsigned char)s[i];
63     return a;
64 }

```

7.7 string/pam.cpp

63 回文自动机

什么时候用

在线维护不同回文子串、最长回文后缀及回文出现次数。字符集为小写字母，建树与空间均为 $O(n)$ 。

调用顺序与接口

- PAM p; p.build(s) 从整串重建; s 带前导空格。也可 p.set(n) 预留容量, 再逐字 p.extend(c), 返回当前最长回文后缀结点。
- 0 是长度 0 的根, 1 是长度 -1 的根; 真正的回文结点为 2..tot, 不同回文数为 tot-1。lst 是当前最长回文后缀。
- len[u] 为长度, fail[u] 为最长真回文后缀; num[u] 为该回文的非空回文后缀个数。因此每次加入后的 num[lst] 是当前前缀末尾的回文数。
- pos[u] 为首次出现的右端点, 首次区间是 [pos[u]-len[u]+1, pos[u]]。cnt=p.count() 返回每个回文的总出现次数, 可重叠; 根的次数置 0。

最小示例 (放入 main):

```
1 PAM p; p.build(" ababa");
2 auto cnt=p.count();
3 assert(p.tot-1==5 && p.len[p.lst]==5);
4 assert(p.num[p.lst]==3 && cnt[p.lst]==1);
5 int u=p.ch[1]['a'-'a']; assert(cnt[u]==3);
6 p.extend('b'); cnt=p.count(); // still safe after count()
7 assert(p.len[p.lst]==5 && cnt[u]==3);
```

使用边界

siz 只记录结点作为最长后缀的次数, 不能直接作为总出现次数。count 在副本上沿 fail 累加, 单次 $O(n)$, 可以重复调用, 也能继续 extend; 不要每加入一个字符都调用 count。所有回文出现次数之和用 ll 保存。空串写作一个空格。

完整代码

```
1 struct PAM{
2     int n=0,tot=1,lst=0;
3     string s;
4     vector<array<int,26>> ch;
5     vector<int> len,fail,num,pos,siz;
6     PAM(){set();}
7     void set(int m=0){
8         assert(0<=m&&m<INT_MAX-2);n=0;tot=1;lst=0;s=" ";
9         ch.assign(2,{});len={0,-1};fail={1,1};
10        num.assign(2,0);pos.assign(2,0);siz.assign(2,0);
11        s.reserve(m+1);ch.reserve(m+2);len.reserve(m+2);fail.reserve(m+2);
12        num.reserve(m+2);pos.reserve(m+2);siz.reserve(m+2);
13    }
14    int get_fail(int u)const{
15        while(s[n-len[u]-1]!=s[n])u=fail[u];
16        return u;
17    }
18    int extend(char c){
19        assert('a'<=c&&c<='z'&&n<INT_MAX-3);s+=c;n++;c-='a';
20        int u=get_fail(lst);
21        if(!ch[u][c]){
22            int v=++tot;
23            ch.push_back({});len.push_back(len[u]+2);
```

```

24     fail.push_back(len[v]==1?0:ch[get_fail(fail[u])][c]);
25     num.push_back(num[fail[v]]+1);pos.push_back(n);siz.push_back(0);
26     ch[u][c]=v;
27 }
28 lst=ch[u][c];siz[lst]++;return lst;
29 }
30 void build(string S){
31     assert(!S.empty()&&S[0]!=' ' &&S.size()<INT_MAX-2);
32     set((int)S.size()-1);
33     for(int i=1;i<(int)S.size();i++)extend(S[i]);
34 }
35 vector<int> count()const{
36     vector<int> cnt=siz;
37     for(int u=tot;u>=2;u--)cnt[fail[u]]+=cnt[u];
38     cnt[0]=cnt[1]=0;return cnt;
39 }
40 };

```

7.8 string/manacher.cpp

64 Manacher 回文半径

什么时候用

线性求出所有奇数、偶数长度回文的最大半径，并以 $O(1)$ 判断某个子串是否回文。

调用顺序与接口

- Manacher t; t.build(s); s 带前导空格，实际下标 1..n; set() 清空。
- d1[i] 包含中心字符：最长奇回文为 [i-d1[i]+1,i+d1[i]-1]，长度 $2*d1[i]-1$ 。
- d2[i] 的中心在 i-1 与 i 之间：最长偶回文为 [i-d2[i],i+d2[i]-1]，长度 $2*d2[i]$ ，半径可以为 0。
- ask(l,r) 判断非空闭区间是否回文，要求 $1 \leq l \leq r \leq n$ 。所有回文出现次数为各位置 d1+d2 的总和，不是不同回文数。

最小示例（放入 main）：

```

1 Manacher t; t.build(" abba");
2 assert(t.d2[3]==2 && t.d1[2]==1);
3 assert(t.ask(1,4) && t.ask(2,3));
4 assert(!t.ask(1,3));
5 long long cnt=0;
6 for(int i=1;i<=t.n;i++)cnt+=t.d1[i]+t.d2[i];
7 assert(cnt==6);

```

使用边界

时间、空间均为 $O(n)$ ；不插入分隔字符，可处理任意字节。奇偶半径含义不同，不能直接把 d1 当成回文长度。空串允许 build，但没有可查询的非空区间。求最长回文取所有 $2*d1[i]-1$ 与 $2*d2[i]$ 的最大值，空串答案为 0。

完整代码

```

1 struct Manacher{
2     int n=0;
3     vector<int> d1,d2;
4     void set(){n=0;d1.clear();d2.clear();}
5     void build(const string &s){
6         assert(!s.empty()&&s[0]!=' ' &&s.size()<INT_MAX/2);

```

```

7   n=(int)s.size()-1;d1.assign(n+1,0);d2.assign(n+1,0);
8   for(int i=1,l=1,r=0;i<=n;i++){
9       int k=i>r?1:min(d1[l+r-i],r-i+1);
10      while(i-k>=1&&i+k<=n&&s[i-k]==s[i+k])k++;
11      d1[i]=k--;
12      if(i+k>r)l=i-k,r=i+k;
13  }
14  for(int i=1,l=1,r=0;i<=n;i++){
15      int k=i>r?0:min(d2[l+r-i+1],r-i+1);
16      while(i-k-1>=1&&i+k<=n&&s[i-k-1]==s[i+k])k++;
17      d2[i]=k;
18      if(i+k-1>r)l=i-k,r=i+k-1;
19  }
20 }
21 bool ask(int l,int r)const{
22     assert(1<=l&&l<=r&&r<=n);int k=r-l+1;
23     return k&1?d1[(l+r)/2]>=k/2+1:d2[(l+r+1)/2]>=k/2;
24 }
25 };

```

7.9 string/runs.cpp

65 Runs 极大周期子串

什么时候用

枚举所有长度至少为最小周期两倍、且不能保持该周期向左或向右扩展的子串。返回若干三元组 (l, r, p) ，其中 p 必须是最小周期。

调用顺序与接口

先抄 string/sa.cpp 和 basic/ST table (linear).cpp，再抄本文件。调用 `Runs t; auto a=t.build(s)`，返回 `vector<array<int,3>>`，依次为 l, r, p 。输入有前导空格，区间为 1 下标闭区间，结果按三元组字典序排序并去重。空串、单字符及无重复的串返回空向量。

最小示例（放入 main）：

```

1 Runs t; auto a=t.build(" ababab");
2 assert((a==vector<array<int,3>>{{1,6,2}}));
3 a=t.build(" aabaa");
4 assert((a==vector<array<int,3>>{{1,2,1},{4,5,1}}));
5 assert(t.build(" abc").empty());

```

使用边界

实际字符限字节 1..255, 0 留给 SA 哨兵。采用两种字母序下的最长 Lyndon 前缀，加正反串 LCP 向两侧扩展；LCP 使用四毛子，不使用哈希。源码版时间 $O(n \log n)$ （最后排序），空间 $O(n)$ ；考场倍增 SA 也满足此界。Runs 不包括所有周期区间，只给出极大区间；长度不必是 p 的倍数，例如 ababa 返回 (1,5,2)。

完整代码

```

1 struct Runs{
2 private:
3     struct LCP{
4         int n=0;
5         vector<int> rk;
6         Linear_RMQ<int> st;
7         void build(const string &s){

```

```

8     SA t;t.build(s,256);n=t.n;rk=move(t.rk);
9     vector<int> h(n+1);
10    for(int i=1,k=0;i<=n;i++){
11        int r=rk[i];if(r==1){k=0;continue;}
12        int j=t.sa[r-1];
13        while(i+k<=n&&j+k<=n&&s[i+k]==s[j+k])k++;
14        h[r]=k;if(k)k--;
15    }
16    st.build(h);
17 }
18 int ask(int i,int j)const{
19     if(i>n||j>n)return 0;
20     if(i==j)return n-i+1;
21     int l=rk[i],r=rk[j];if(l>r)swap(l,r);
22     return st.ask(l+1,r);
23 }
24 };
25 public:
26 vector<array<int,3>> build(const string &s)const{
27     assert(!s.empty()&&s[0]!=' ' &&s.size()<(INT_MAX-1024)/2);
28     int n=(int)s.size()-1;
29     for(int i=1;i<=n;i++)assert((unsigned char)s[i]>0);
30     vector<array<int,3>> ans;if(n<2)return ans;
31     LCP a,b;a.build(s);
32     string t=s;reverse(t.begin()+1,t.end());b.build(t);
33     vector<int> st;st.reserve(n);
34     for(int op=0;op<2;op++){
35         st.clear();
36         for(int i=n;i>=1;i--){
37             while(!st.empty()){
38                 int j=st.back(),k=a.ask(i,j);
39                 // Reversing the alphabet does not reverse the prefix rule.
40                 if(j+k>n)break;
41                 bool less=(unsigned char)s[i+k]<(unsigned char)s[j+k];
42                 if(less==bool(op))break;
43                 st.pop_back();
44             }
45             if(!st.empty()){
46                 int j=st.back(),p=j-i;
47                 int l=b.ask(n-i+2,n-j+2),r=a.ask(i,j);
48                 if(l+r>p)ans.push_back({i-l,j+r-1,p});
49             }
50             st.push_back(i);
51         }
52     }
53     sort(ans.begin(),ans.end());ans.erase(unique(ans.begin(),ans.end()),
54         ans.end());
55     return ans;
56 }
57 };

```

7.10 string/lyndon.cpp

66 Lyndon 分解

什么时候用

用 Duval 将串唯一分解为字典序不增的 Lyndon 串。Lyndon 串严格小于自己的每个非空真后缀，相同因子可以连续出现。

调用顺序与接口

`auto a=lyndon(s)`; `s` 带前导空格。返回 `vector<pair<int,int>>`，各项为一个因子的 1 下标闭区间 `[l,r]`，按原串从左到右排列且恰好覆盖全串。需要内容时用 `s.substr(l,r-l+1)`。

最小示例（放入 main）：

```
1 string s=" ababab"; auto a=lyndon(s);
2 assert((a==vector<pair<int,int>>{{1,2},{3,4},{5,6}}));
3 assert((lyndon(" cba")==vector<pair<int,int>>{{1,1},{2,2},{3,3}}));
4 assert(lyndon(" ").empty());
```

使用边界

时间 $O(n)$ ，除返回区间外额外空间 $O(1)$ 。比较按 `unsigned char` 的字节顺序，可处理任意字节；不要求小写字母。同样的因子不会合并，`ababab` 分为三个 `ab`，而 `aaaa` 分为四个 `a`。空串返回空向量。本接口是整串分解，不是每个位置的最长 Lyndon 前缀数组。

完整代码

```
1 vector<pair<int,int>> lyndon(const string &s){
2     assert(!s.empty()&&s[0]!=' ' &&s.size()<INT_MAX);
3     int n=(int)s.size()-1;vector<pair<int,int>> ans;
4     for(int i=1;i<=n;){
5         int j=i+1,k=i;
6         while(j<=n&&(unsigned char)s[k]<=(unsigned char)s[j]){
7             k=s[k]==s[j]?k+1:i;j++;
8         }
9         int p=j-k;
10        while(i<=k){ans.emplace_back(i,i+p-1);i+=p;}
11    }
12    return ans;
13 }
```

8 计算几何

8.1 geometry/geometry-main.cpp

72 整数二维几何基础

什么时候用

点坐标和叉积均为 `ll`，提供凸包、面积与基本相交判断；需要另定义 `ld=long double`。

调用顺序与接口

- `Point(x,y)`：点向量加减；`a*b` 是叉积，`a^b` 是点积。
- `convex(points)` 求凸包，`convex_area(hull)` 返回两倍面积。
- `on_segment`、`strict_intersect`、`nonstrict_intersect` 分别判断在线段上、严格相交、含端点相交。

粘贴本文件之前：

```
1 using ld=long double;
```

最小示例（放入 main）：

```
1 vector<Point> a{{0,0},{2,0},{2,2},{0,2}};
2 auto h=convex(a); assert(convex_area(h)==8);
3 assert(on_segment(Point(1,0),Point(0,0),Point(2,0)));
```

使用边界

`convex` 自动排序去重。`min_cross` 要求顶点按凸包环序排列且没有共线中间点，支持旋转起点、顺逆时针， $O(\log n)$ 。射线包含端点，零方向按单点处理；退化三角形按三条边处理。坐标叉积、点积仍须保证不溢出 `ll`。`Point` 输出两个坐标，中间有空格；`output` 用于调试。

完整代码

```
1 const ld pi=acos(-1),dx=1e-9;
2 struct Point{
3     ll x,y;
4     friend istream& operator >>(istream &i,Point &p){i>>p.x>>p.y;return i;}
5     friend ostream& operator <<(ostream &o,const Point &p){o<<p.x<<' '<<p.y;
6         return o;}
7     // void read(){cin>>x>>y;}
8     // void output(){cout<<x<<' '<<y<<'\n';}
9     void output(string s=""){cerr<<"Point "<<s<<": ("<<x<<","<<y<<")<<endl;}
10    double arg(){return atan2(y,x);}
11    Point (ll _x=0,ll _y=0){x=_x,y=_y;}
12 };
13
14 inline bool operator==(const Point &x,const Point &y){return
15     x.x==y.x&&x.y==y.y;}
16 inline bool operator!=(const Point &x,const Point &y){return
17     x.x!=y.x||x.y!=y.y;}
18
19 inline bool operator<(const Point &x,const Point &y){return
20     (x.x!=y.x?x.x<y.x:x.y<y.y);}
21 inline Point operator-(const Point &x,const Point &y){return Point(x.x-y.x,
22     x.y-y.y);}
23 inline Point operator+(const Point &x,const Point &y){return Point(x.x+y.x,
24     x.y+y.y);}
25 inline void operator-=(Point &x,const Point &y){x=x-y;}
```

```

26 inline void operator+=(Point &x,const Point &y){x=x+y;}
27 inline Point operator*(const Point &x,ll y){return Point(x.x*y,x.y*y);}
28 inline void operator*=(Point &x,ll y){x=x*y;}
29 inline Point operator*(ll y,const Point &x){return Point(x.x*y,x.y*y);}
30 inline void operator*=(ll y,Point &x){x=x*y;}
31
32 inline ll operator*(const Point &x,const Point &y){
33     // return OX * OY * sin(XOY) = 2 * area(OXY)
34     return x.x*y.y-x.y*y.x;
35 }
36 inline ll operator^(const Point &x,const Point &y){
37     // return OX * OY * cos(XOY)
38     return x.x*y.x+x.y*y.y;
39 }
40
41 int sgn(ll x){return (x==0?0:(x>0?1:-1));}
42
43 inline int sgn(const Point &P){
44     return P.y<0?-1:P.y>0|P.x<0?1:0;
45 }
46 inline bool cmp(const Point &x,const Point &y){
47     if(sgn(x)!=sgn(y)) return sgn(x)<sgn(y);
48     return x*y>0;
49 }
50 inline ll area(const Point &x,const Point &y,const Point &z){
51     // return triangle XYZ area **times 2**
52     return abs((x-y)*(x-z));
53 }
54
55 inline ld dist(const Point &x){return sqrt((ld)x.x*x.x+(ld)x.y*x.y);}
56 inline ll dist2(const Point &x){return x.x*x.x+x.y*x.y;}
57
58 inline int dir(const Point &x,const Point &y,const Point &z){
59     // return is YX clockwise direction to YZ
60     return sgn((x-y)*(y-z));
61 }
62 inline int suf(int x,int n){return (x+1==n?0:x+1);}
63 inline int pre(int x,int n){return (x==0?n-1:x-1);}
64 ll convec_area(const vector<Point> &a){
65     // return convex A area **times 2**
66     ll s=0;int n=a.size();
67     for(int i=0;i<n;i++) s+=area(a[0],a[i],a[suf(i,n)]);
68     return s;
69 }
70 vector<Point> convex(vector<Point> a){
71     sort(a.begin(),a.end());a.erase(unique(a.begin(),a.end()),a.end());
72     int n=a.size();
73     if(n<=2)return a;
74     vector<Point> b;int s=0,t=0;
75     for(int i=2;i-->0;s=t,reverse(a.begin(),a.end())){
76         for(Point p:a){
77             while(t>=s+2&&dir(b[t-2],b[t-1],p)>=0) t--,b.pop_back();
78             b.push_back(p);t++;
79         }
80         b.pop_back();t--;

```

```

81 }
82 return b;
83 }
84 inline bool on_segment(const Point &X,const Point &A,const Point &B){
85     // check if X on segment AB
86     if((A-B)*(X-B)!=0) return false;
87     return min(A.x,B.x)<=X.x&&X.x<=max(A.x,B.x)&&
88         min(A.y,B.y)<=X.y&&X.y<=max(A.y,B.y);
89 }
90 inline bool strict_intersect(const Point &A,const Point &B,
91     const Point &C,const Point &D){
92     // check if AB,CD (except endpoint)strictly intersect
93     return dir(A,C,D)*dir(B,C,D)==-1&&dir(C,A,B)*dir(D,A,B)==-1;
94 }
95 inline bool nonstrict_intersect(const Point &A,const Point &B,
96     const Point &C,const Point &D){
97     // check if AB,CD (include endpoints) intersect
98     return (dir(A,C,D)*dir(B,C,D)==-1&&dir(C,A,B)*dir(D,A,B)==-1)||
99     on_segment(A,C,D)||on_segment(B,C,D)||on_segment(C,A,B)||on_segment(D,A,B);
100 }
101 inline bool parallel(Point A,Point B,
102     Point C,Point D){
103     // check if AB // CD
104     B-=A,D-=C;A-=C;C=-1*D;
105     return sgn(B*C)==0&&sgn(A*B)!=0;
106 }
107 inline bool ray_intersect(Point A,Point B,Point C,Point D){
108     // closed rays; a zero direction is a single point
109     Point u=B-A,v=D-C,w=C-A;
110     if(u==Point())return v==Point()?A==C:((A-C)*v==0&&((A-C)^v)>=0);
111     if(v==Point())return w*u==0&&(w^u)>=0;
112     ll d=u*v;
113     if(!d)return w*u==0&&((u^v)>0||(w^u)>=0);
114     return sgn(w*v)*sgn(d)>=0&&sgn(w*u)*sgn(d)>=0;
115 }
116 inline bool in_triangle(const Point &P,const Point &A,
117     const Point &B,const Point &C){
118     // check if P in triangle ABC
119     // P may lie on bound
120     if(!area(A,B,C))return on_segment(P,A,B)||on_segment(P,B,C)||on_segment(P,
121         C,A);
122     return area(A,B,C)==area(P,A,B)+area(P,B,C)+area(P,C,A);
123 }
124 template<typename F> int convex_min(int n,F val){
125     assert(n>0);
126     auto low=[&](int i){return val(i)<=val((i+n-1)%n)&&val(i)<=val((i+1)%n);};
127     if(low(0))return 0;
128     int l=0,r=n;
129     while(l+1<r){
130         int mid=(l+r)>>1;
131         if(low(mid))return mid;
132         bool a=val((l+1)%n)>=val(l),b=val((mid+1)%n)>=val(mid);
133         if(a!=b){if(a)l=mid;else r=mid;}
134         else if((val(mid)>val(l))==a)l=mid;
135         else r=mid;

```

```
136     }
137     return r%n;
138 }
139 inline ll min_cross(const vector<Point> &I,Point X){
140     // convex polygon in cyclic order, without collinear intermediate points
141     auto val=[&](int i){return I[i]*X;};
142     return val(convex_min(I.size(),val));
143 }
144 inline ld min_cross(const vector<Point> &I,ld x,ld y){
145     auto val=[&](int i){return I[i].x*x+I[i].y*y;};
146     return val(convex_min(I.size(),val));
147 }
```